

Week 07 — Interactions II & Quantities of Interest

Standard errors of marginal effects, continuous moderators, moderators in DAGs, and expected values, first differences and predicted values by bootstrap

AUTHORS

Johann-Friedrich Salzmann

Joshua Elias Schmidt Rodrigues

DATE

2026-10-22

Topics:

Package management 2
Where we left off 2
Data: election results and county structure 2
The standard error of a marginal effect 5
Interactions with a continuous moderator 11
Moderators in DAGs 20
Expected vs. predicted values 21
Quantities of interest 24
Concluding remarks 45
Try it: the marginal effect of age 45
Try it: a prediction for Mannheim 46
Try it: first differences in turnout 47

Package management

New this week is `marginaleffects`, which computes marginal effects, predictions and their uncertainty for almost any model. `infer` from Week 6 does the bootstrap, `ggdag` and `dagitty` from Week 5 draw the DAG, and `patchwork` puts several plots together.

```
if (!require(pacman)) install.packages("pacman")
```

MESSAGE

```
Loading required package: pacman
```

```
p_load(tidyverse, broom, marginaleffects, infer, ggdag, dagitty, patchwork)
set.seed(707)
```

Where we left off

Week 6 let the turnout slope differ between East and West. The Eastern slope $\hat{\beta}_1 + \hat{\beta}_3$ had no standard error of its own. Today we give it one, move on to a moderator that is continuous, and ask what a moderator means causally. The second half turns coefficients into **quantities of interest**: expected AfD shares, differences between them, and predictions for single counties, each with its uncertainty.

Each section starts with its **quantity of interest**, in most cases an **estimand**: the quantity we want to know, defined with the true coefficients β. What we compute from the data is an **estimate** of it, written with a hat.

Data: election results and county structure

The GERDA counties from Weeks 5 and 6 again, without Berlin. This week we use the 2021 election, because the county covariates below exist for 2021, but not yet for 2025:

```
gerda <- read_csv("data/federal_cty_harm.csv")
```

MESSAGE

```
Rows: 4000 Columns: 127
-- Column specification --------------------------------------------------------
Delimiter: ","
chr    (2): county_code, state
dbl  (124): election_year, eligible_voters, number_voters, valid_votes, inva...
date   (1): election_date

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

New are the county covariates from the INKAR database that the `gerda` package provides (you met them in the Week 6 homework). The file in `data/` was created with the commented-out line. We take the election year:

```
# p_load(gerda); gerda_covariates() |> write_csv("data/cty_covariates.csv")
covariates <- read_csv("data/cty_covariates.csv")
```

MESSAGE

```
Rows: 11200 Columns: 32
-- Column specification --------------------------------------------------------
Delimiter: ","
chr  (1): county_code
dbl (31): year, share_65plus, median_age, share_foreign, share_female, gdp_p...

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
covariates_2021 <- covariates |>
  filter(year == 2021) |>
  select(county_code, median_age)
covariates_2021
```

OUTPUT

```
# A tibble: 400 x 2
   county_code median_age
   <chr>            <dbl>
 1 01001             42.0
 2 01002             41.9
 3 01003             45.0
 4 01004             44.5
 5 01051             46.3
 6 01053             44.9
 7 01054             46.1
 8 01055             48.4
 9 01056             44.7
10 01057             47.3
# i 390 more rows
```

```r
east_codes <- c("12", "13", "14", "15", "16")

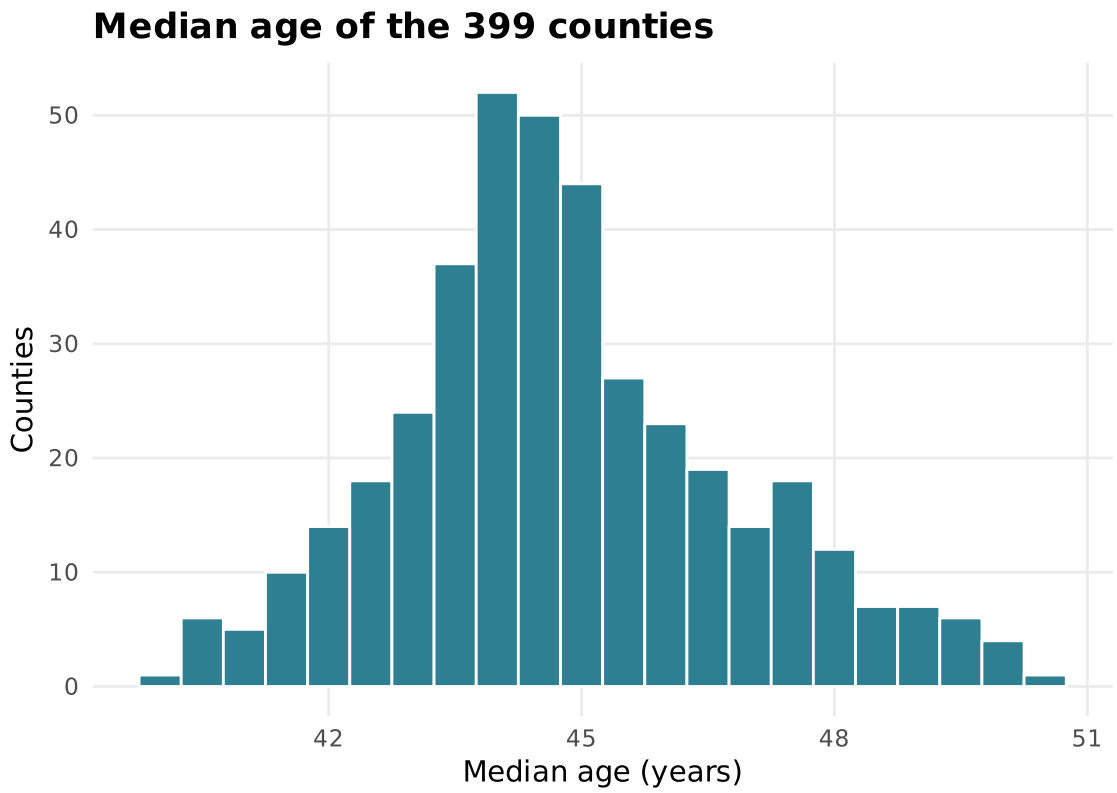
counties <- gerda |>
  filter(
    election_year == 2021,
    state != "11"
  ) |>
  mutate(
    east   = if_else(state %in% east_codes, 1, 0),
    region = if_else(east == 1, "East", "West")
  ) |>
  select(county_code, state, east, region, afd, turnout) |>
  mutate(across(c(afd, turnout), ~ .x * 100)) |>
  left_join(covariates_2021, by = "county_code")
counties
```

OUTPUT

```
# A tibble: 399 x 7
   county_code state  east region   afd turnout median_age
   <chr>       <chr> <dbl> <chr>  <dbl>   <dbl>      <dbl>
 1 01001       01        0 West    5.42    74.0       42.0
 2 01002       01        0 West    5.02    76.2       41.9
 3 01003       01        0 West    6.63    72.4       45.0
 4 01004       01        0 West    9.38    67.9       44.5
 5 01051       01        0 West    8.80    76.0       46.3
 6 01053       01        0 West    8.35    78.9       44.9
 7 01054       01        0 West    5.23    77.7       46.1
 8 01055       01        0 West    7.03    77.8       48.4
 9 01056       01        0 West    6.87    80.3       44.7
10 01057       01        0 West    6.32    81.2       47.3
# i 389 more rows
```

`median_age` is the median age of a county's population in years. It is roughly normally distributed, so we use it as it is:

```
counties |>
  ggplot(aes(x = median_age)) +
  geom_histogram(binwidth = 0.5, fill = course_primary, color = "white") +
  labs(
    title = "Median age of the 399 counties",
    x     = "Median age (years)",
    y     = "Counties"
  ) +
  theme_intror()
```




```
region_colors <- c(
  "West" = course_primary,
  "East" = "darkorange"
)
```

The standard error of a marginal effect

Quantity of interest (estimand): the marginal effect of turnout on the AfD share in the West and in the East, β_1 and $\beta_1 + \beta_3$. New today is the standard error of its estimate.

Week 6's interaction model, now fitted to the 2021 election:

$$\text{AfD}_i = \beta_0 + \beta_1 \text{Turnout}_i + \beta_2 \text{East}_i + \beta_3 \text{Turnout}_i \cdot \text{East}_i + \varepsilon_i$$

```r
interaction_model <- lm(afd ~ turnout * east, data = counties)
interaction_model |>
  tidy()
```

OUTPUT

```
# A tibble: 4 x 5
  term         estimate std.error statistic      p.value
  <chr>           <dbl>     <dbl>     <dbl>        <dbl>
1 (Intercept)    20.0      3.46       5.79  0.0000000141
2 turnout        -0.144    0.0448    -3.20  0.00148
3 east           -4.60     7.72      -0.596 0.552
4 turnout:east    0.230    0.104      2.21  0.0278
```

```r
interaction_beta_1 <- interaction_model |>
  tidy() |>
  filter(term == "turnout") |>
  pull(estimate)
interaction_beta_3 <- interaction_model |>
  tidy() |>
  filter(term == "turnout:east") |>
  pull(estimate)
interaction_p_3 <- interaction_model |>
  tidy() |>
  filter(term == "turnout:east") |>
  pull(p.value)
```

In general, with an outcome Y, a predictor X and a **moderator** Z,

$$Y_i = \beta_0 + \beta_1 X_i + \beta_2 Z_i + \beta_3 X_i Z_i + \varepsilon_i$$

the marginal effect of X is the slope of Y in X at a given value of Z:

$$\frac{\partial Y}{\partial X} = \beta_1 + \beta_3 Z$$

Here X is turnout and Z is the East dummy: the Western slope is the marginal effect at $Z = 0$, the Eastern slope the one at $Z = 1$.

The standard error of an estimate is the estimated standard deviation of its sampling distribution. The lecture writes it as $\hat{\sigma}$ with the estimate as subscript, so $\mathrm{SE}\big(\hat{\theta}\big) = \hat{\sigma}_{\hat{\theta}}$ (not to be confused with $\hat{\sigma}$ alone, the residual standard deviation). The estimated marginal effect, with a wide hat, $\widehat{\frac{\partial Y}{\partial X}} = \hat{\beta}_1 + \hat{\beta}_3 Z$, combines two estimates, and both are uncertain. Its standard error depends on their variances and on how they vary together, their covariance:

$$\mathrm{SE}\left(\widehat{\frac{\partial Y}{\partial X}}\right) = \hat{\sigma}_{\widehat{\frac{\partial Y}{\partial X}}} = \sqrt{\widehat{\mathrm{Var}}\big(\hat{\beta}_1\big) + Z^2 \widehat{\mathrm{Var}}\big(\hat{\beta}_3\big) + 2Z \widehat{\mathrm{Cov}}\big(\hat{\beta}_1, \hat{\beta}_3\big)}$$

`tidy()` only gives us the standard errors of single coefficients. The formula needs three ingredients: the variances of $\hat{\beta}_1$ and $\hat{\beta}_3$, and their covariance.

The variance–covariance matrix

All three ingredients sit in the estimated **variance–covariance matrix** $\hat{\boldsymbol{V}}$ of the coefficients: variances on the diagonal (shaded), covariances off it. The three entries our formula needs each have their own color:

$$\hat{\boldsymbol{V}} = \begin{pmatrix} \widehat{\mathrm{Var}}\left(\hat{\beta}_0\right) & \widehat{\mathrm{Cov}}\left(\hat{\beta}_0, \hat{\beta}_1\right) & \widehat{\mathrm{Cov}}\left(\hat{\beta}_0, \hat{\beta}_2\right) & \widehat{\mathrm{Cov}}\left(\hat{\beta}_0, \hat{\beta}_3\right) \\ \widehat{\mathrm{Cov}}\left(\hat{\beta}_1, \hat{\beta}_0\right) & \widehat{\mathrm{Var}}\left(\hat{\beta}_1\right) & \widehat{\mathrm{Cov}}\left(\hat{\beta}_1, \hat{\beta}_2\right) & \widehat{\mathrm{Cov}}\left(\hat{\beta}_1, \hat{\beta}_3\right) \\ \widehat{\mathrm{Cov}}\left(\hat{\beta}_2, \hat{\beta}_0\right) & \widehat{\mathrm{Cov}}\left(\hat{\beta}_2, \hat{\beta}_1\right) & \widehat{\mathrm{Var}}\left(\hat{\beta}_2\right) & \widehat{\mathrm{Cov}}\left(\hat{\beta}_2, \hat{\beta}_3\right) \\ \widehat{\mathrm{Cov}}\left(\hat{\beta}_3, \hat{\beta}_0\right) & \widehat{\mathrm{Cov}}\left(\hat{\beta}_3, \hat{\beta}_1\right) & \widehat{\mathrm{Cov}}\left(\hat{\beta}_3, \hat{\beta}_2\right) & \widehat{\mathrm{Var}}\left(\hat{\beta}_3\right) \end{pmatrix}$$

The matrix is symmetric: $\widehat{\mathrm{Cov}}\left(\hat{\beta}_1, \hat{\beta}_3\right)$ and $\widehat{\mathrm{Cov}}\left(\hat{\beta}_3, \hat{\beta}_1\right)$ are the same number. `vcov()` returns it as a matrix. We turn it into a tibble with one row per coefficient:

```
interaction_vcov <- interaction_model |>
  vcov() |>
  as_tibble(rownames = "term")
interaction_vcov
```

OUTPUT

```
# A tibble: 4 x 5
  term         `(Intercept)`  turnout    east `turnout:east`
  <chr>                <dbl>    <dbl>   <dbl>          <dbl>
1 (Intercept)         11.9    -0.155   -11.9          0.155
2 turnout             -0.155   0.00201   0.155       -0.00201
3 east               -11.9     0.155    59.6         -0.803
4 turnout:east         0.155  -0.00201  -0.803        0.0109
```

The matrix feeds more than one formula. Ours takes the three colored entries:

$$\mathrm{SE}\left(\widehat{\frac{\partial Y}{\partial X}}\right) = \sqrt{\widehat{\mathrm{Var}}\left(\hat{\beta}_1\right) + Z^2 \widehat{\mathrm{Var}}\left(\hat{\beta}_3\right) + 2Z \widehat{\mathrm{Cov}}\left(\hat{\beta}_1, \hat{\beta}_3\right)}$$

Just for the overview: the standard errors of single coefficients in `tidy()` are the square roots of the shaded diagonal. We don't need them here.

$$\mathrm{SE}\left(\hat{\beta}_j\right) = \sqrt{\widehat{\mathrm{Var}}\left(\hat{\beta}_j\right)}$$

For example, the turnout slope:

$$\mathrm{SE}\left(\hat{\beta}_1\right) = \sqrt{\widehat{\mathrm{Var}}\left(\hat{\beta}_1\right)}$$

We pull out the two variances and the covariance:

```r
var_beta_1 <- interaction_vcov |>
  filter(term == "turnout") |>
  pull(turnout)
var_beta_3 <- interaction_vcov |>
  filter(term == "turnout:east") |>
  pull(`turnout:east`)
cov_beta_1_3 <- interaction_vcov |>
  filter(term == "turnout") |>
  pull(`turnout:east`)
cov_beta_1_3
```

OUTPUT

```
[1] -0.002011239
```

The covariance is negative: a sample that gives a steeper Western slope tends to give a larger difference to the East.

Computing the standard error

We compute both regions' marginal effects in one tibble, with Z as the column `east`:

$$\widehat{\frac{\partial Y}{\partial X}} = \frac{\widehat{\partial \text{ AfD}}}{\partial \text{ Turnout}} = \hat{\beta}_1 + \hat{\beta}_3 \text{ East}$$

```r
me_regions <- tibble(
  region = c("West", "East"),
  east   = c(0, 1)
) |>
  mutate(me = interaction_beta_1 + interaction_beta_3 * east)
```

Then the standard error formula from above, with $Z = \text{East}$:

$$\text{SE}\left(\widehat{\frac{\partial Y}{\partial X}}\right) = \text{SE}\left(\frac{\widehat{\partial \text{ AfD}}}{\partial \text{ Turnout}}\right) = \sqrt{\widehat{\text{Var}}\left(\hat{\beta}_1\right) + \text{East}^2\widehat{\text{Var}}\left(\hat{\beta}_3\right) + 2 \text{ East } \widehat{\text{Cov}}\left(\hat{\beta}_1, \hat{\beta}_3\right)}$$

For the East, East $= 1$:

$$\text{SE}\left(\frac{\widehat{\partial \text{ AfD}}}{\partial \text{ Turnout}} \mid \text{East} = 1\right) = \sqrt{\widehat{\text{Var}}\left(\hat{\beta}_1\right) + \widehat{\text{Var}}\left(\hat{\beta}_3\right) + 2\widehat{\text{Cov}}\left(\hat{\beta}_1, \hat{\beta}_3\right)}$$

In the West, East $= 0$ leaves only the standard error of $\hat{\beta}_1$ in `tidy()`:

$$\text{SE}\left(\frac{\widehat{\partial \text{ AfD}}}{\partial \text{ Turnout}} \mid \text{East} = 0\right) = \sqrt{\widehat{\text{Var}}\left(\hat{\beta}_1\right)}$$

```r
me_regions <- me_regions |>
  mutate(se = sqrt(var_beta_1 + east^2 * var_beta_3 + 2 * east * cov_beta_1_3))
me_regions
```

OUTPUT

```
# A tibble: 2 x 4
  region  east      me     se
  <chr>  <dbl>   <dbl>  <dbl>
1 West       0 -0.144  0.0448
2 East       1  0.0865 0.0940
```

The 95% confidence interval uses the t distribution with $n - k - 1$ degrees of freedom, as in Week 6:

$$\text{CI}_{95\%} = \frac{\partial \widehat{\text{AfD}}}{\partial \text{ Turnout}} \pm t_{n-k-1,0.975} \cdot \text{SE}\left(\frac{\partial \widehat{\text{AfD}}}{\partial \text{ Turnout}}\right)$$

```r
interaction_df <- interaction_model |>
  glance() |>
  pull(df.residual)

me_regions <- me_regions |>
  mutate(
    lower = me - qt(0.975, interaction_df) * se,
    upper = me + qt(0.975, interaction_df) * se
  )
me_regions
```

OUTPUT

```
# A tibble: 2 x 6
  region  east      me     se   lower   upper
  <chr>  <dbl>   <dbl>  <dbl>   <dbl>   <dbl>
1 West       0 -0.144  0.0448 -0.232  -0.0554
2 East       1  0.0865 0.0940 -0.0984  0.271
```

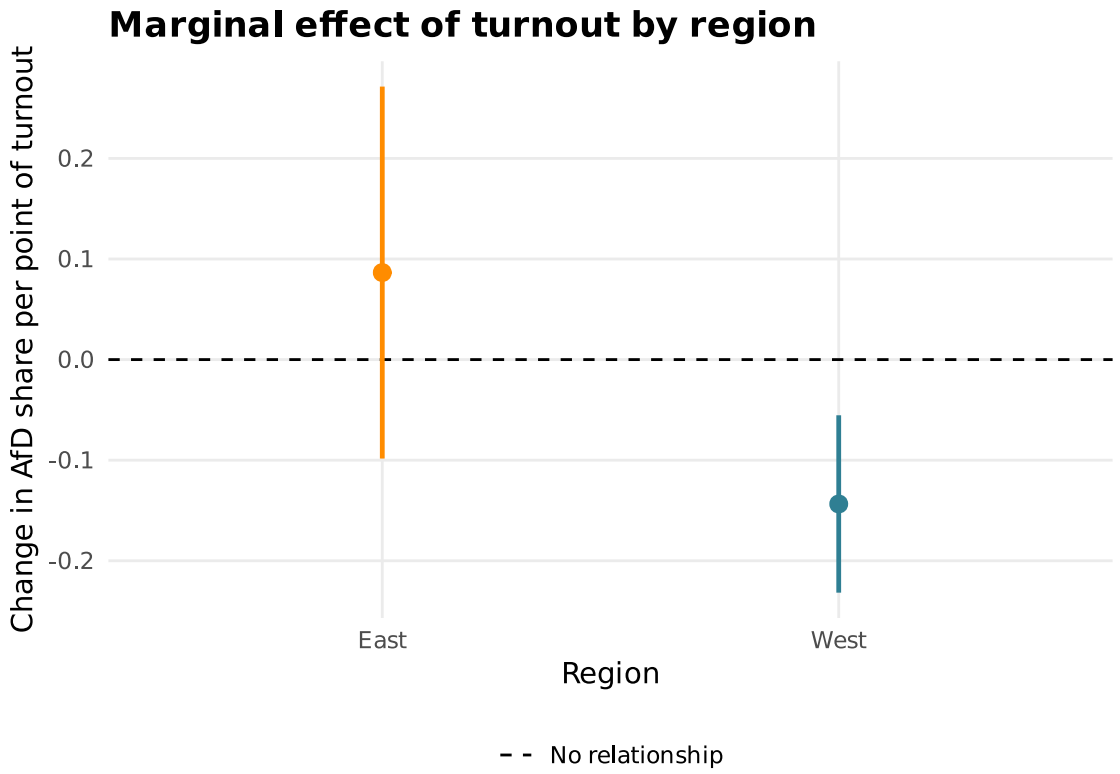
Both regions at a glance:

Region	ME	SE	ME formula	SE formula
West	-0.144	0.045	$\begin{cases} \hat{\beta}_1 & \text{East} = 0 \\ \hat{\beta}_1 + \hat{\beta}_3 & \text{East} = 1 \end{cases}$	$\begin{cases} \sqrt{\widehat{\text{Var}}(\hat{\beta}_1)} & \text{East} = 0 \\ \sqrt{\widehat{\text{Var}}(\hat{\beta}_1) + \widehat{\text{Var}}(\hat{\beta}_3) + 2\widehat{\text{Cov}}(\hat{\beta}_1, \hat{\beta}_3)} & \text{East} = 1 \end{cases}$
East	0.087	0.094		

In the West, AfD support is lower where turnout is higher: the 95% CI $[-0.23, -0.06]$ excludes zero. In the East, there is no clear relationship: the CI $[-0.1, 0.27]$ includes zero.

The same as a plot:

```r
me_regions |>
  ggplot(aes(x = region, y = me, color = region)) +
  geom_hline(aes(yintercept = 0, linetype = "No relationship")) +
  geom_pointrange(aes(ymin = lower, ymax = upper), linewidth = 1) +
  scale_color_manual(values = region_colors, guide = "none") +
  scale_linetype_manual(values = c("No relationship" = "dashed")) +
  labs(
    title    = "Marginal effect of turnout by region",
    x        = "Region",
    y        = "Change in AfD share per point of turnout",
    color    = NULL,
    linetype = NULL
  ) +
  theme_intror()
```



Comparison with `slopes()`

`slopes()` from `marginaleffects` computes the marginal effect of the variable in `variables` at the rows given in `newdata`. `datagrid()` builds these rows; variables we don't name are set to their mean:

```r
me_regions_slopes <- slopes(
  interaction_model,
  variables = "turnout",
  newdata   = datagrid(east = c(0, 1)),
  df        = "residual" # without it: Normal instead of t with the model's df
)
me_regions_slopes
```

OUTPUT

```
 east Estimate Std. Error     t Pr(>|t|)   S   2.5 %  97.5 %  Df
    0  -0.1435     0.0448 -3.20  0.00148 9.4 -0.2317 -0.0554 395
    1   0.0865     0.0940  0.92  0.35813 1.5 -0.0984  0.2714 395

Term: turnout
Type: response
Comparison: dY/dX
```

`df = "residual"` makes `marginaleffects` use the t distribution with the model's residual degrees of freedom, as we did by hand. Its default is the Normal, because the package serves many models (logit, for example) for which only the Normal approximation is justified. With 395 degrees of freedom the two are practically the same. In a small sample they are not: with 20 counties and four coefficients, the Normal interval would come out about 8% too narrow.

Side by side:

```r
me_regions_slopes_values <- me_regions_slopes |>
  as_tibble() |>
  select(east, me_slopes = estimate, se_slopes = std.error)

me_regions |>
  select(region, east, me, se) |>
  left_join(me_regions_slopes_values, by = "east")
```

OUTPUT

```
# A tibble: 2 x 6
  region  east      me     se me_slopes se_slopes
  <chr>  <dbl>   <dbl>  <dbl>     <dbl>     <dbl>
1 West       0 -0.144  0.0448   -0.144     0.0448
2 East       1  0.0865 0.0940    0.0865    0.0940
```

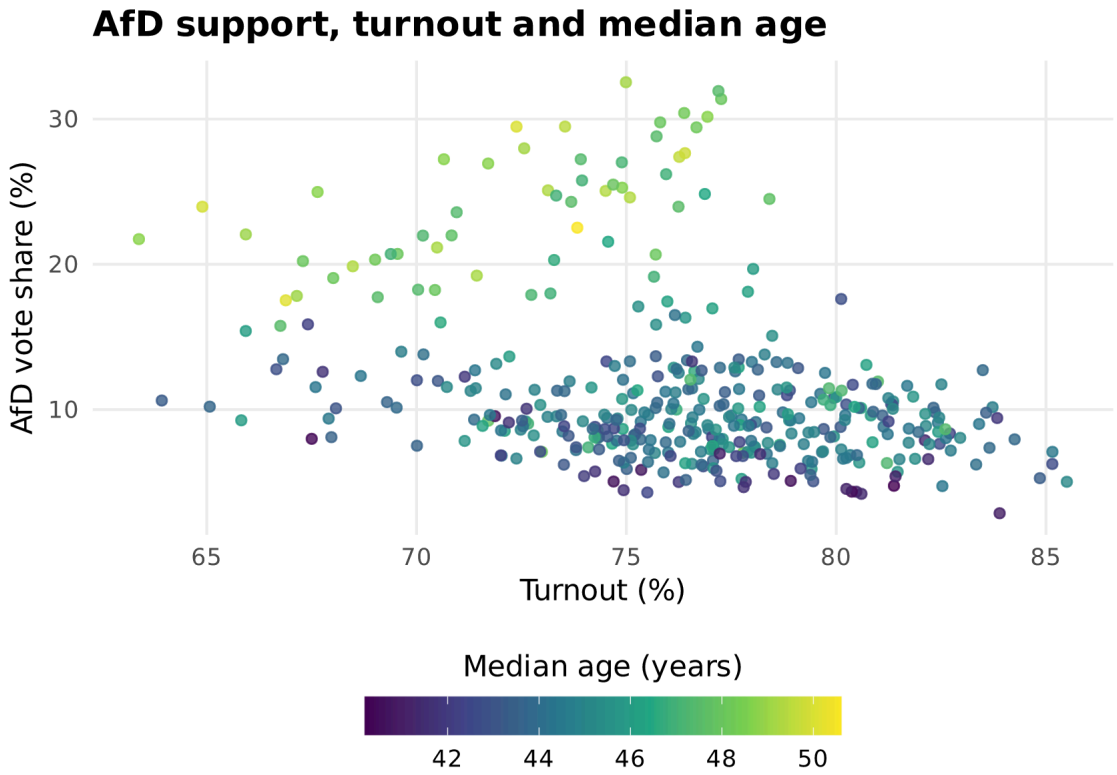
The two agree. In 2021, the Western slope is clearly negative and the Eastern one is not distinguishable from zero. Their difference, $\hat{\beta}_3$, has a p-value of 0.028: the two slopes differ at the 5% level.

Interactions with a continuous moderator

Quantity of interest (estimand): the marginal effect of turnout at every median age, $\beta_1 + \beta_3$ Age.

Does turnout relate to AfD support differently in younger and older counties? Older electorates vote more reliably and differently, so the moderator is now median age, a continuous variable. A first look, with each county colored by its median age:

```
counties |>
  ggplot(aes(x = turnout, y = afd, color = median_age)) +
  geom_point(alpha = 0.8) +
  scale_color_viridis_c(breaks = seq(40, 50, 2)) +
  labs(
    title = "AfD support, turnout and median age",
    x     = "Turnout (%)",
    y     = "AfD vote share (%)",
    color = "Median age (years)"
  ) +
  theme_intror() +
  theme(legend.key.width = unit(1.5, "cm")) +
  guides(color = guide_colorbar(title.position = "top", title.hjust = 0.5))
```



The colors are not mixed at random: the older (yellow) counties sit mostly at higher AfD shares, the younger (purple) ones lower. So median age varies along with AfD support, and also with turnout.

The model, with Z now median age:

$$\text{AfD}_i = \beta_0 + \beta_1 \text{Turnout}_i + \beta_2 \text{Age}_i + \beta_3 \text{Turnout}_i \cdot \text{Age}_i + \varepsilon_i$$

```r
age_model <- lm(afd ~ turnout * median_age, data = counties)
age_model |>
  tidy()
```

OUTPUT

```
# A tibble: 4 x 5
  term                estimate std.error statistic p.value
  <chr>                  <dbl>     <dbl>     <dbl>   <dbl>
1 (Intercept)         -218.       78.4        -2.79 0.00561
2 turnout                1.96      1.05        1.87 0.0616
3 median_age             5.57      1.74        3.19 0.00152
4 turnout:median_age    -0.0496    0.0233     -2.13 0.0339
```

```r
age_beta_0 <- age_model |>
  tidy() |>
  filter(term == "(Intercept)") |>
  pull(estimate)
age_beta_1 <- age_model |>
  tidy() |>
  filter(term == "turnout") |>
  pull(estimate)
age_beta_2 <- age_model |>
  tidy() |>
  filter(term == "median_age") |>
  pull(estimate)
age_beta_3 <- age_model |>
  tidy() |>
  filter(term == "turnout:median_age") |>
  pull(estimate)
```

$$\widehat{\text{AfD}}_i = -218 + 1.96 \cdot \text{Turnout}_i + 5.57 \cdot \text{Age}_i - 0.0496 \cdot \text{Turnout}_i \cdot \text{Age}_i$$

As in Week 6, $\hat{\beta}_1$ is the turnout slope where the moderator is zero, here in a county with a median age of 0 years. It means nothing on its own. In Week 6, the dummy allowed two slopes, one for the West and one for the East. Now every value of age has its own slope: the marginal effect of turnout changes continuously with age.

$$\frac{\partial \widehat{\text{AfD}}}{\partial \text{ Turnout}} = \hat{\beta}_1 + \hat{\beta}_3 \text{ Age}$$

Lines at different ages

A continuous moderator has no natural groups, so we pick values: the 10th, 50th and 90th percentile of median age.

```r
age_levels <- counties |>
  summarize(
    p10 = quantile(median_age, 0.1),
    p50 = quantile(median_age, 0.5),
    p90 = quantile(median_age, 0.9)
  ) |>
  pivot_longer(
    everything(),
    names_to        = "percentile",
    names_prefix    = "p",
    names_transform = as.numeric,
    values_to       = "median_age"
  )
age_levels
```

OUTPUT

```
# A tibble: 3 x 2
  percentile median_age
       <dbl>      <dbl>
1         10       42.4
2         50       44.5
3         90       47.6
```

```r
age_levels_values <- age_levels |>
  pull(median_age)
```

For each age, the predicted AfD share along the observed range of turnout:

$$\widehat{\text{AfD}} = \hat{\beta}_0 + \hat{\beta}_1 \text{ Turnout} + \hat{\beta}_2 \text{ Age} + \hat{\beta}_3 \text{ Turnout} \cdot \text{Age}$$

```r
turnout_min <- counties |>
  summarize(min(turnout)) |>
  pull()
turnout_max <- counties |>
  summarize(max(turnout)) |>
  pull()
turnout_grid <- seq(turnout_min, turnout_max, length.out = 50)
```

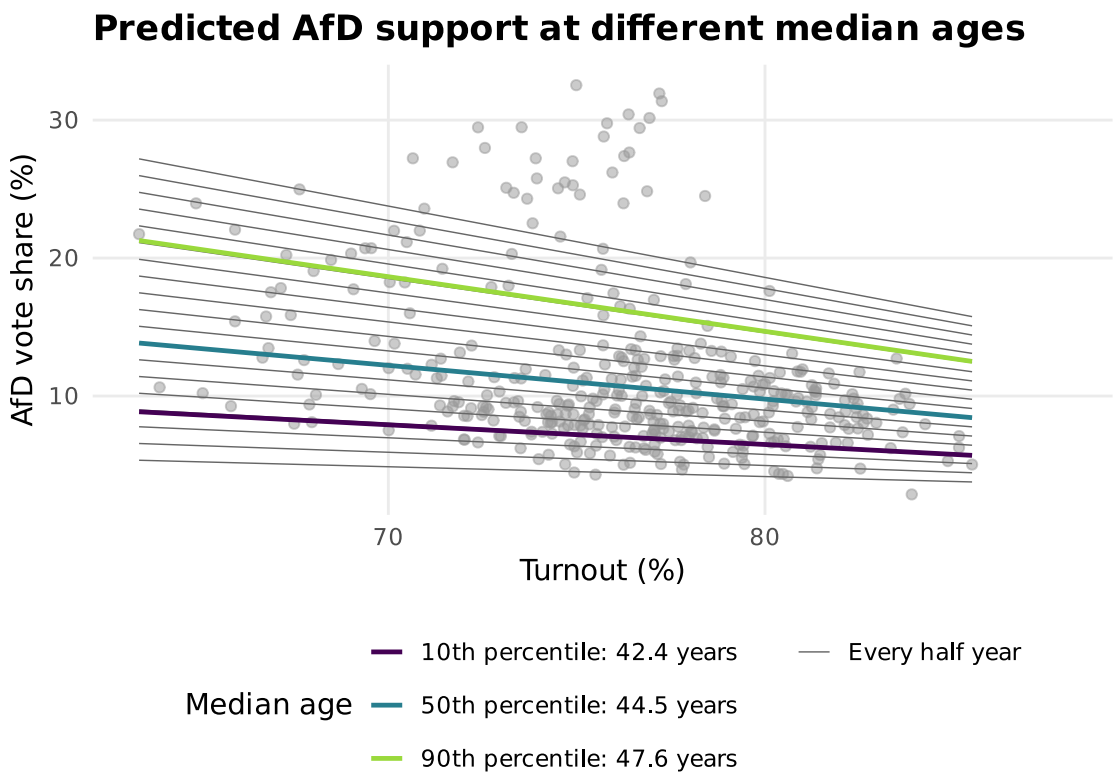
```r
age_lines <- expand_grid(age_levels, turnout = turnout_grid) |>
  mutate(
    afd_hat   = age_beta_0 + age_beta_1 * turnout + age_beta_2 * median_age + age_beta_3
* turnout * median_age,
    age_label = str_glue("{percentile}th percentile: {round(median_age, 1)} years")
  )
```

The same for many more ages, every half year across the observed range:

```r
age_min <- counties |>
  summarize(min(median_age)) |>
  pull()
age_max <- counties |>
  summarize(max(median_age)) |>
  pull()
other_age_step <- 0.5
```

```r
other_age_lines <- expand_grid(
  median_age = seq(ceiling(age_min), floor(age_max), by = other_age_step),
  turnout    = turnout_grid
) |>
   mutate(afd_hat = age_beta_0 + age_beta_1 * turnout + age_beta_2 * median_age +
age_beta_3 * turnout * median_age)
```

```r
age_lines_plot <- ggplot(mapping = aes(x = turnout, y = afd_hat)) +
  geom_point(data = counties, aes(y = afd), color = "grey60", alpha = 0.5) +
   geom_line(data = other_age_lines, aes(group = median_age, linetype = "Every half
year"), color = "grey40", linewidth = 0.3) +
  geom_line(data = age_lines, aes(color = age_label), linewidth = 1) +
  scale_color_viridis_d(end = 0.85) +
  scale_linetype_manual(values = c("Every half year" = "solid")) +
  guides(color = guide_legend(ncol = 1, order = 1)) +
  labs(
    title    = "Predicted AfD support at different median ages",
    x        = "Turnout (%)",
    y        = "AfD vote share (%)",
    color    = "Median age",
    linetype = NULL
  ) +
  theme_intror()
```



The older the county, the more steeply AfD support falls with turnout: the line at the 90th percentile is far steeper than the one at the 10th.

The same with `marginaleffects`

`predictions()` computes the same lines, one expected value per combination of turnout and age in the grid:

```
predictions(
  age_model,
  newdata = datagrid(turnout = turnout_grid, median_age = age_levels_values)
)
```

The marginal effect across ages

Instead of single lines, we plot the marginal effect itself over the whole range of median age, with the standard error from above. The variances and the covariance now come from the age model:

```r
age_vcov <- age_model |>
  vcov() |>
  as_tibble(rownames = "term")

age_var_beta_1 <- age_vcov |>
  filter(term == "turnout") |>
  pull(turnout)
age_var_beta_3 <- age_vcov |>
  filter(term == "turnout:median_age") |>
  pull(`turnout:median_age`)
age_cov_beta_1_3 <- age_vcov |>
  filter(term == "turnout") |>
  pull(`turnout:median_age`)
```

The marginal effect of turnout is zero where $\hat{\beta}_1 + \hat{\beta}_3 \text{ Age} = 0$:

$$\text{Age}_0 = -\frac{\hat{\beta}_1}{\hat{\beta}_3}$$

```r
age_me_zero <- -age_beta_1 / age_beta_3
age_me_zero
```

OUTPUT

```
[1] 39.56426
```

The youngest county has a median age of 40.2 years, so the relationship would only turn positive just outside the data. We extend the age grid down to this point:

```r
age_grid <- seq(min(age_me_zero, age_min), age_max, length.out = 50)
```

The interval as in the East–West case:

$$\frac{\partial \widehat{\text{AfD}}}{\partial \text{ Turnout}} = \hat{\beta}_1 + \hat{\beta}_3 \text{ Age}$$

$$\text{SE}\left(\frac{\partial \widehat{\text{AfD}}}{\partial \text{ Turnout}}\right) = \sqrt{\widehat{\text{Var}}\left(\hat{\beta}_1\right) + \text{Age}^2 \widehat{\text{Var}}\left(\hat{\beta}_3\right) + 2 \text{ Age } \widehat{\text{Cov}}\left(\hat{\beta}_1, \hat{\beta}_3\right)}$$

$$\text{CI}_{95\%} = \frac{\partial \widehat{\text{AfD}}}{\partial \text{ Turnout}} \pm t_{n-k-1,0.975} \cdot \text{SE}\left(\frac{\partial \widehat{\text{AfD}}}{\partial \text{ Turnout}}\right)$$

```
age_df <- age_model |>
  glance() |>
  pull(df.residual)

age_me_curve <- tibble(median_age = age_grid) |>
  mutate(
    me    = age_beta_1 + age_beta_3 * median_age,
     se     = sqrt(age_var_beta_1 + median_age^2 * age_var_beta_3 + 2 * median_age
* age_cov_beta_1_3),
    lower = me - qt(0.975, age_df) * se,
    upper = me + qt(0.975, age_df) * se
  )
age_me_curve
```

OUTPUT

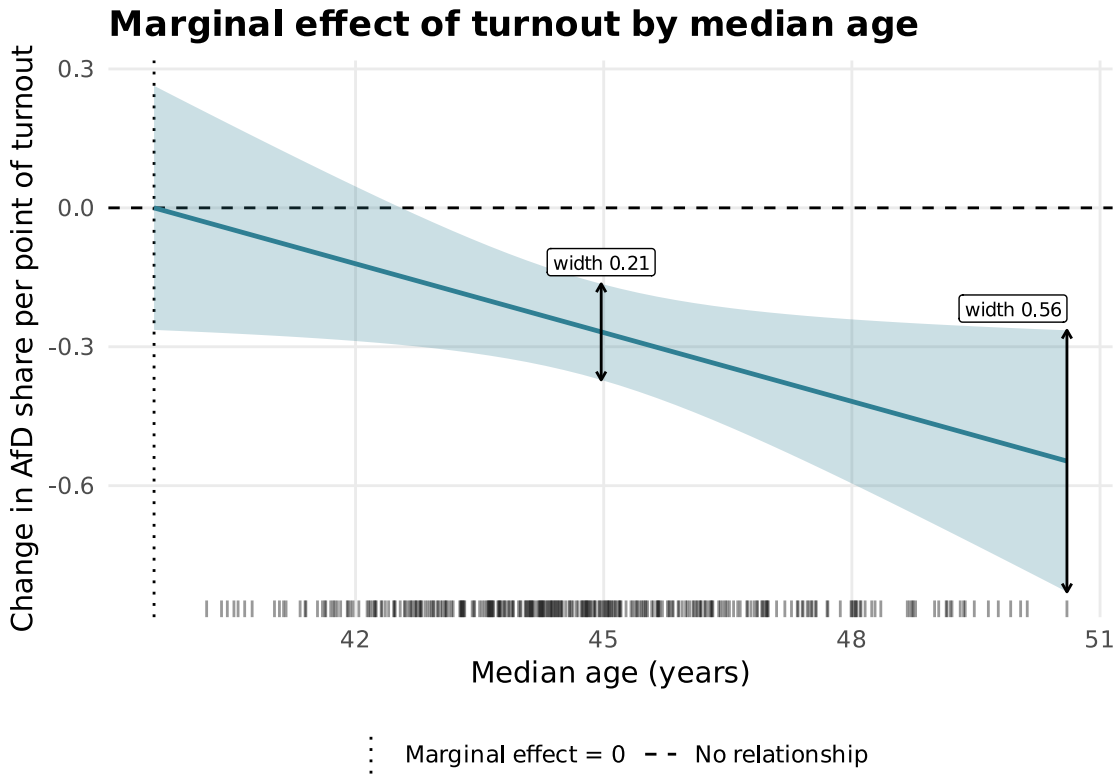
```
# A tibble: 50 x 5
   median_age      me     se  lower  upper
        <dbl>   <dbl>  <dbl>  <dbl>  <dbl>
 1       39.6  0      0.134  -0.264 0.264
 2       39.8 -0.0112 0.129  -0.265 0.243
 3       40.0 -0.0223 0.125  -0.267 0.222
 4       40.2 -0.0335 0.120  -0.269 0.202
 5       40.5 -0.0446 0.115  -0.271 0.182
 6       40.7 -0.0558 0.110  -0.273 0.161
 7       40.9 -0.0670 0.106  -0.275 0.141
 8       41.1 -0.0781 0.101  -0.277 0.121
 9       41.4 -0.0893 0.0969 -0.280 0.101
10       41.6 -0.100  0.0926 -0.282 0.0815
# i 40 more rows
```

The rug at the bottom marks the counties' median ages: the estimate is only backed by data where they lie. The arrows mark the width of the band at two ages:

```
narrow_band <- age_me_curve |>
  slice_min(se)
wide_band <- age_me_curve |>
  slice_max(median_age)

band_widths <- bind_rows(narrow_band, wide_band) |>
  mutate(
    label       = str_glue("width {round(upper - lower, 2)}"),
    label_hjust = if_else(median_age == age_max, 1, 0.5)
  )
```

```
age_me_curve |>
  ggplot(aes(x = median_age)) +
  geom_hline(aes(yintercept = 0, linetype = "No relationship")) +
  geom_vline(aes(xintercept = age_me_zero, linetype = "Marginal effect = 0")) +
  geom_ribbon(aes(ymin = lower, ymax = upper), fill = course_primary, alpha = 0.25) +
  geom_line(aes(y = me), color = course_primary, linewidth = 1) +
  geom_segment(
    data  = band_widths,
    aes(xend = median_age, y = lower, yend = upper),
    arrow = arrow(ends = "both", length = unit(0.15, "cm"))
  ) +
  geom_label(data = band_widths, aes(y = upper, label = label, hjust = label_hjust),
size = 3, vjust = -0.3) +
  geom_rug(data = counties, aes(x = median_age), alpha = 0.4) +
  scale_linetype_manual(values = c("No relationship" = "dashed", "Marginal effect = 0"
= "dotted")) +
  labs(
    title    = "Marginal effect of turnout by median age",
    x        = "Median age (years)",
    y        = "Change in AfD share per point of turnout",
    linetype = NULL
  ) +
  theme_intror()
```



In the youngest counties, the interval includes zero: there, turnout and AfD support are not clearly related. The older the county, the more negative the slope. At the oldest county's median age, one more point of turnout goes along with 0.55 points less AfD support. From a median age of about 42.7 years on, the whole interval lies below zero.

The band is narrowest at 45 years, near the average county, and widens towards both ends. In the standard error, the uncertain $\hat{\beta}_3$ is multiplied by age: the further we move from where most counties lie, the more an error in how the slope changes with age adds up.

`slopes()` gives the same marginal effects and standard errors at the three chosen ages:

```
slopes(
  age_model,
  variables = "turnout",
  newdata   = datagrid(median_age = age_levels_values),
  df        = "residual"
)
```

OUTPUT

```
 median_age Estimate Std. Error     t Pr(>|t|)    S  2.5 %   97.5 %  Df
       42.4   -0.143     0.0770 -1.86   0.0642  4.0 -0.294  0.00846 395
       44.5   -0.245     0.0533 -4.59   <0.001 17.3 -0.349 -0.13974 395
       47.6   -0.396     0.0821 -4.83   <0.001 19.0 -0.558 -0.23500 395

Term: turnout
Type: response
Comparison: dY/dX
```

The same with `marginaleffects`

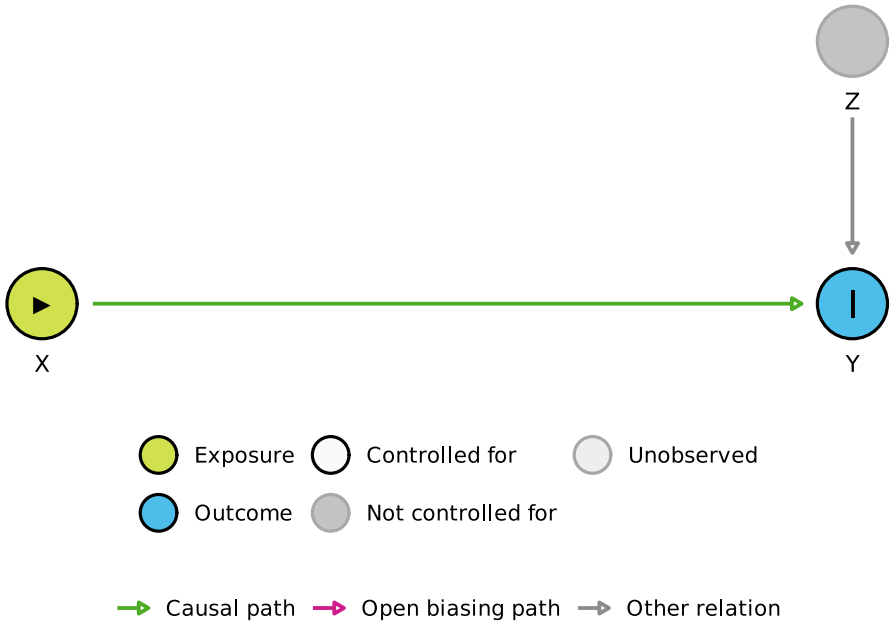
With the whole age grid as `newdata`, `slopes()` gives every point of the curve above, with the same standard errors and t-based intervals as `age_me_curve`:

```
slopes(
  age_model,
  variables = "turnout",
  newdata   = datagrid(median_age = age_grid),
  df        = "residual"
)
```

Moderators in DAGs

Quantity of interest (estimand): the causal effect of X on Y at a given value of Z. The DAG tells us which variables a model needs to estimate it.

A minimal DAG with two causes of Y:



This DAG says $Y = f(X, Z, U)$, with U for everything else, and nothing about the form of f. That includes $Y = \beta_1 X + \beta_2 Z$ just as well as $Y = \beta_1 X + \beta_2 Z + \beta_3 XZ$. A DAG is **nonparametric**: whether X's effect depends on Z is part of f, not of the graph. Three consequences:

- **A moderator needs no symbol of its own.** Any other cause of Y can modify the effect of X. The arrow from Z *onto* the arrow $X \to Y$, common in textbooks, is a sketch, not a DAG: arrows connect variables, never arrows, and `dagitty` cannot read it.
- **A DAG cannot tell you whether there is an interaction.** That is the model's job, as above. Adjustment sets still come from the DAG: we need them to read the effect of X at each value of Z causally.
- **Moderator and mediator are two different roles.** A mediator lies on the path $X \to M \to Y$ and carries part of X's effect. A moderator changes how large the effect of X is. One variable can play both roles, if X also causes it, but neither role implies the other.

Why "moderator" at all? *Interaction* is the statistical term, and the product term treats X and Z the same way: it lets Z change the slope of X, and X the slope of Z. Which of the two we call the **moderator** is our choice, made by the question we ask. We asked about turnout's effect, so East and median age moderate it. The Try-it on the marginal effect of age turns this around.

Causal inference draws one more line. **Effect modification** means the effect of intervening on X differs across values of Z; Z only sorts the cases. An **interaction** in the strict causal sense means the effect of intervening on *both* X and Z together differs from the sum of intervening on each alone. Our age example is effect modification at most: nobody intervenes on a county's median age.

Expected vs. predicted values

Quantity of interest (estimand): the expected AfD share of a county with a given median age, $E(\text{AfD} \mid \text{Age}) = \beta_0 + \beta_1$ Age. A predicted value is a quantity of interest but not an estimand in this sense: it aims at a single county's AfD share, which stays random even with the true β.

Take a regression with a single predictor. The model has two parts, a systematic one and the error term:

$$\text{AfD}_i = \beta_0 + \beta_1 \text{Age}_i + \varepsilon_i$$

The error term follows a Normal distribution:

$$\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$$

```
simple_model <- lm(afd ~ median_age, data = counties)
simple_sigma <- simple_model |>
  glance() |>
  pull(sigma)
simple_sigma
```

OUTPUT

```
[1] 4.35965
```

`augment()` (or base R's `predict()`) returns the systematic part only, the **expected value** of a county with this age:

$$E(\text{AfD} \mid \text{Age}) = \hat{\beta}_0 + \hat{\beta}_1 \ \text{Age}$$

To **predict** a county the way the model says the data come about, we add a draw of the error term:

$$\widetilde{\text{AfD}} = \hat{\beta}_0 + \hat{\beta}_1 \ \text{Age} + \tilde{\varepsilon}, \quad \tilde{\varepsilon} \sim \mathcal{N}(0, \hat{\sigma}^2)$$

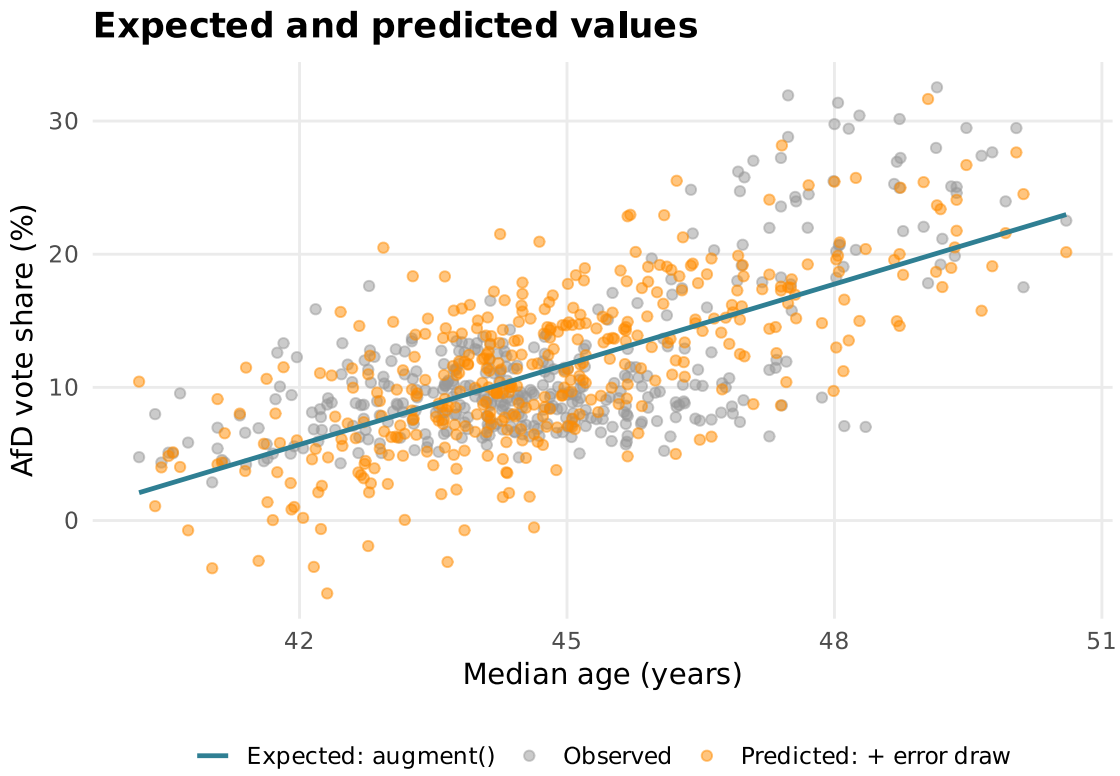
$$\widetilde{\text{AfD}} = -78.65 + 2.01 \cdot \text{Age} + \tilde{\varepsilon}, \quad \tilde{\varepsilon} \sim \mathcal{N}(0, 4.36^2)$$

```
simple_values <- simple_model |>
  augment() |>
  mutate(predicted = .fitted + rnorm(n(), 0, simple_sigma))
simple_values
```

OUTPUT

```
# A tibble: 399 x 9
     afd median_age .fitted  .resid    .hat .sigma  .cooksd .std.resid predicted
   <dbl>      <dbl>   <dbl>   <dbl>   <dbl>  <dbl>    <dbl>      <dbl>     <dbl>
 1  5.42       42.0    5.79  -0.364 0.00734   4.37  2.59e-5    -0.0837     0.198
 2  5.02       41.9    5.51  -0.488 0.00784   4.37  4.99e-5    -0.112      2.81
 3  6.63       45.0   11.6   -5.00  0.00252   4.36  1.66e-3    -1.15      14.7
 4  9.38       44.5   10.7   -1.34  0.00257   4.36  1.23e-4    -0.309     15.7
 5  8.80       46.3   14.4   -5.58  0.00391   4.36  3.22e-3    -1.28      10.8
 6  8.35       44.9   11.5   -3.16  0.00251   4.36  6.61e-4    -0.725      6.99
 7  5.23       46.1   13.9   -8.69  0.00351   4.34  7.02e-3    -2.00      22.9
 8  7.03       48.4   18.5  -11.4   0.0103    4.33  3.61e-2    -2.64      20.4
 9  6.87       44.7   11.1   -4.19  0.00252   4.36  1.17e-3    -0.963      5.91
10  6.32       47.3   16.3   -9.97  0.00625   4.34  1.65e-2    -2.29      24.1
# i 389 more rows
```

```
simple_values |>
  ggplot(aes(x = median_age)) +
  geom_point(aes(y = afd, color = "Observed"), alpha = 0.5) +
  geom_point(aes(y = predicted, color = "Predicted: + error draw"), alpha = 0.5) +
  geom_line(aes(y = .fitted, color = "Expected: augment()"), linewidth = 1) +
  scale_color_manual(values = c(
    "Observed"                = "grey60",
    "Predicted: + error draw" = "darkorange",
    "Expected: augment()"     = course_primary
  )) +
  labs(
    title = "Expected and predicted values",
    x     = "Median age (years)",
    y     = "AfD vote share (%)",
    color = NULL
  ) +
  theme_intror()
```



The expected values all lie on the line. The predicted values scatter around it like the observed counties do. Some of them, for the youngest counties, fall below 0%: the Normal error term doesn't know that vote shares can't be negative. That is a limit of the linear model, not a mistake in the code.

The same with `marginaleffects`

`predictions()` returns the same expected values as `.fitted`, one per county, with a confidence interval each:

```
predictions(simple_model)
```

Two kinds of uncertainty enter such statements:

- **Estimation uncertainty**: we don't know the true β. It shrinks with more data. Expected values, first differences and marginal effects carry only this one.
- **Fundamental uncertainty**: even with the true β, a single county deviates from its expectation by $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$. Predicted values carry both.

Careful with the names: R's `predict()` and `marginaleffects`' `predictions()` both return *expected* values. Neither adds the error term. Where each tool gives which:

	Expected value	Predicted value
`predict()` (base R)	`predict(model, newdata)`, interval with `interval = "confidence"`	interval with `interval = "prediction"`, no single draws
`augment()`	column `.fitted`, interval with `interval = "confidence"`	interval with `interval = "prediction"`; single draws: `.fitted + rnorm(n(), 0, sigma)`
`marginaleffects`	`predictions()`, with its confidence interval	no built-in way for `lm()`

So far we used $\hat{\beta}$ as if it were the truth. The next section adds the estimation uncertainty.

Quantities of interest

From here on we use the age model to answer substantive questions. The lecture's **quantities of interest** (estimands) are functions of the coefficients:

- **Marginal effect** $\frac{\partial Y}{\partial X}$: how Y changes with X, all else equal, as above.
- **Expected value** $E(Y \mid X)$: the average Y for given values of the predictors.
- **First difference** $E(Y \mid X_1) - E(Y \mid X_2)$: the difference between two expected values. Elsewhere it is called a *predictive comparison* (Gelman & Hill) or a *contrast*; `marginaleffects` calls it a *comparison* (`comparisons()`).
- **Predicted value** $\hat{Y} \mid X$: a single new observation for given values of the predictors.

Bootstrapping the coefficients

The lecture fits the model once and then draws coefficients from the multivariate Normal $\mathcal{MVN}\left(\hat{\beta}, \hat{\boldsymbol{V}}\right)$. We take the bootstrap from Week 3 instead: draw 399 counties with replacement, fit the model again, repeat. No Normal distribution and no $\hat{\boldsymbol{V}}$ are needed. The steps:

1. Draw bootstrap samples: 399 counties with replacement, many times.

2. Fit the model on each sample. The coefficients form their sampling distribution.
3. Choose covariate values, a **scenario**.
4. Calculate the quantity of interest (estimand) with each bootstrap sample's coefficients.
5. Summarize its distribution.

Step 1. As in Week 3, `expand_grid()` builds one block of rows per bootstrap sample, numbered by `sample_id`, here the whole `counties` tibble each time. `slice_sample()` then draws the counties of each sample with replacement:

```
n_boot <- 1000

boot_samples <- expand_grid(
  sample_id = 1:n_boot,
  counties
) |>
  group_by(sample_id) |>
  slice_sample(prop = 1, replace = TRUE) |>
  ungroup()
boot_samples
```

OUTPUT

```
# A tibble: 399,000 x 8
   sample_id county_code state  east region   afd turnout median_age
       <int> <chr>       <chr> <dbl> <chr>  <dbl>   <dbl>      <dbl>
 1         1 09778       09        0 West   11.8     80.9       43.8
 2         1 09176       09        0 West    9.42    83.8       41.9
 3         1 05758       05        0 West    9.63    75.5       44.6
 4         1 05166       05        0 West    5.97    77.1       45.5
 5         1 09677       09        0 West    8.47    82.4       45.9
 6         1 08417       08        0 West   13.0     74.7       44.6
 7         1 06435       06        0 West   11.2     75.9       44.2
 8         1 03355       03        0 West    6.50    77.1       43.6
 9         1 09179       09        0 West    6.22    83.3       44.0
10         1 16066       16        1 East   25.5     74.7       48.0
# i 398,990 more rows
```

Step 2. We fit the model on each bootstrap sample. This gives the bootstrap coefficients:

$$\tilde{\beta}^{(r)} = \text{OLS estimate on bootstrap sample } r, \quad r = 1, ..., R$$

`rowwise()` works through one replicate r per row: filter its sample (`sample_id == replicate`), fit and tidy. Within `rowwise()`, each row's result must be a single row, so `pivot_wider()` puts the four coefficients side by side; the `mutate()` argument has no name, so they become ordinary columns. `rename()` gives the intercept the name `infer` uses below, and `pivot_longer()` at the end gives one row per replicate and coefficient:

```r
boot_coefs <- tibble(replicate = 1:n_boot) |>
  rowwise() |>
  mutate(
    boot_samples |>
      filter(sample_id == replicate) |>
      lm(afd ~ turnout * median_age, data = _) |>
      tidy() |>
      select(term, estimate) |>
      pivot_wider(names_from = term, values_from = estimate)
  ) |>
  ungroup() |>
  rename(intercept = `(Intercept)`) |>
  pivot_longer(-replicate, names_to = "term", values_to = "estimate")
boot_coefs
```

OUTPUT

```
# A tibble: 4,000 x 3
   replicate term                estimate
       <int> <chr>                  <dbl>
 1         1 intercept          -164.
 2         1 turnout               1.22
 3         1 median_age            4.34
 4         1 turnout:median_age   -0.0329
 5         2 intercept          -136.
 6         2 turnout               0.867
 7         2 median_age            3.76
 8         2 turnout:median_age   -0.0254
 9         3 intercept          -213.
10         3 turnout               1.98
# i 3,990 more rows
```

The same in one compressed pipe with `infer`: `generate()` shuffled in Week 6, with `type = "bootstrap"` it resamples, and `fit()` estimates the model once per replicate. It returns the same long shape as `boot_coefs`, whose names we chose to match: `replicate`, and `intercept` for the intercept.

```
boot_coefs_infer <- counties |>
  specify(afd ~ turnout * median_age) |>
  generate(reps = n_boot, type = "bootstrap") |>
  fit()
boot_coefs_infer
```

OUTPUT

```
# A tibble: 4,000 x 3
# Groups:   replicate [1,000]
   replicate term                estimate
       <int> <chr>                  <dbl>
 1         1 intercept          -374.
 2         1 turnout               4.03
 3         1 median_age            9.00
 4         1 turnout:median_age   -0.0953
 5         2 intercept          -349.
 6         2 turnout               3.88
 7         2 median_age            8.47
 8         2 turnout:median_age   -0.0924
 9         3 intercept          -231.
10         3 turnout               2.12
# i 3,990 more rows
```

`marginaleffects` bootstraps the coefficients too: `hypotheses()` returns them, `inferences()` resamples, and `get_draws()` gives one row per replicate and coefficient. We bring it into the same shape:

```
boot_coefs_marginaleffects <- hypotheses(age_model) |>
  inferences(method = "boot", R = n_boot) |>
  get_draws() |>
  as_tibble() |>
  select(replicate = drawid, term, estimate = draw) |>
  mutate(term = if_else(term == "(Intercept)", "intercept", term))
boot_coefs_marginaleffects
```

OUTPUT

```
# A tibble: 4,000 x 3
   replicate term                estimate
   <fct>     <chr>                  <dbl>
 1 1         intercept          -346.
 2 1         turnout               3.56
 3 1         median_age            8.43
 4 1         turnout:median_age   -0.0853
 5 2         intercept          -198.
 6 2         turnout               1.56
 7 2         median_age            5.04
 8 2         turnout:median_age   -0.0396
 9 3         intercept          -236.
10 3         turnout               2.14
# i 3,990 more rows
```

The standard deviation of each coefficient across replicates is its bootstrap standard error:

$$\mathrm{SE}_{\mathrm{boot}(\hat{\beta}_j)} = \mathrm{sd}\left(\tilde{\beta}_j^{(1)}, \ldots, \tilde{\beta}_j^{(R)}\right)$$

```
boot_se <- boot_coefs |>
  group_by(term) |>
  summarize(se_boot = sd(estimate))

infer_se <- boot_coefs_infer |>
  group_by(term) |>
  summarize(se_infer = sd(estimate))

marginaleffects_se <- boot_coefs_marginaleffects |>
  group_by(term) |>
  summarize(se_marginaleffects = sd(estimate))

age_se <- age_model |>
  tidy() |>
  select(term, se_ols = std.error) |>
  mutate(term = if_else(term == "(Intercept)", "intercept", term))

age_se |>
  left_join(boot_se, by = "term") |>
  left_join(infer_se, by = "term") |>
  left_join(marginaleffects_se, by = "term")
```

OUTPUT

```
# A tibble: 4 x 5
  term                se_ols se_boot se_infer se_marginaleffects
  <chr>                <dbl>   <dbl>    <dbl>              <dbl>
1 intercept          78.4    77.2     79.1               77.7
2 turnout             1.05    1.04     1.07               1.05
3 median_age          1.74    1.74     1.78               1.75
4 turnout:median_age  0.0233  0.0235   0.0241             0.0236
```

All three bootstraps agree, up to the randomness of the draws. Our own standard errors are −2 to 1% larger than the OLS ones. The OLS formula assumes that all counties scatter equally around the regression line. The bootstrap only uses the counties themselves: where some scatter more than others, its standard errors come out larger.

From here on we work with one row per replicate and one column per coefficient:

```r
boot_betas <- boot_coefs |>
  pivot_wider(names_from = term, values_from = estimate) |>
  rename(
    beta_0 = intercept,
    beta_1 = turnout,
    beta_2 = median_age,
    beta_3 = `turnout:median_age`
  )
boot_betas
```

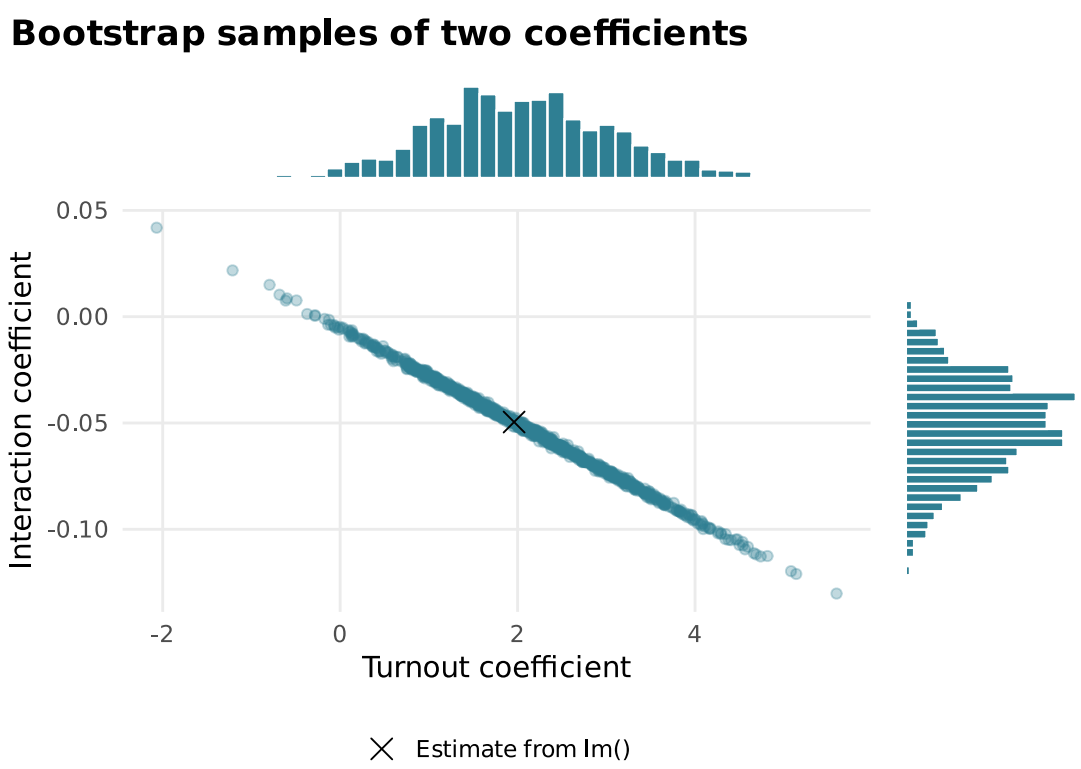
OUTPUT

```
# A tibble: 1,000 x 5
   replicate beta_0 beta_1 beta_2  beta_3
       <int>  <dbl>  <dbl>  <dbl>   <dbl>
 1         1  -164.  1.22    4.34 -0.0329
 2         2  -136.  0.867   3.76 -0.0254
 3         3  -213.  1.98    5.43 -0.0499
 4         4  -178.  1.34    4.54 -0.0341
 5         5  -211.  1.84    5.37 -0.0465
 6         6  -144.  0.854   3.89 -0.0244
 7         7  -243.  2.21    5.97 -0.0533
 8         8  -256.  2.45    6.37 -0.0600
 9         9  -180.  1.49    4.70 -0.0389
10        10  -376.  4.06    9.17 -0.0976
# i 990 more rows
```

The bootstrap samples also show the covariance from above. Where $\tilde{\beta}_1$ comes out larger, $\tilde{\beta}_3$ comes out more negative:

```r
lm_estimate <- tibble(
  beta_1 = age_beta_1,
  beta_3 = age_beta_3
)

boot_betas_plot <- boot_betas |>
  ggplot(aes(x = beta_1, y = beta_3)) +
  geom_point(alpha = 0.3, color = course_primary) +
  geom_point(data = lm_estimate, aes(shape = "Estimate from lm()"), size = 4) +
  scale_shape_manual(values = c("Estimate from lm()" = 4)) +
  labs(
    title = "Bootstrap samples of two coefficients",
    x     = "Turnout coefficient",
    y     = "Interaction coefficient",
    shape = NULL
  ) +
  theme_intror()
```



The histograms on the sides show each coefficient on its own: their standard deviations are the bootstrap standard errors above. Only the cloud in the middle shows how the two move together, the covariance.

Step 3. Our scenario: two counties at the mean turnout, one young (10th percentile of median age), one old (90th percentile).

```
turnout_mean <- counties |>
  summarize(mean(turnout)) |>
  pull()
age_young <- age_levels |>
  filter(percentile == 10) |>
  pull(median_age)
age_old <- age_levels |>
  filter(percentile == 90) |>
  pull(median_age)
```

Expected values and first differences

Quantity of interest (estimand): the expected AfD share at mean turnout in the young and in the old county, and the first difference between them.

Step 4. Step 3 fixed the inputs. Step 4 turns each bootstrap sample's coefficients into the quantity of interest (estimand). For each bootstrap sample r, the expected AfD share in the young county:

$$E\big(\text{AfD} \mid \overline{\text{Turnout}}, \text{Age}_{\text{young}}\big)^{(r)} = \tilde{\beta}_0^{(r)} + \tilde{\beta}_1^{(r)}\overline{\text{Turnout}} + \tilde{\beta}_2^{(r)}\text{Age}_{\text{young}} + \tilde{\beta}_3^{(r)}\overline{\text{Turnout}} \cdot \text{Age}_{\text{young}}$$

the same in the old county:

$$E\big(\text{AfD} \mid \overline{\text{Turnout}}, \text{Age}_{\text{old}}\big)^{(r)} = \tilde{\beta}_0^{(r)} + \tilde{\beta}_1^{(r)}\overline{\text{Turnout}} + \tilde{\beta}_2^{(r)}\text{Age}_{\text{old}} + \tilde{\beta}_3^{(r)}\overline{\text{Turnout}} \cdot \text{Age}_{\text{old}}$$

and the first difference between them:

$$\mathrm{FD}^{(r)} = E\big(\mathrm{AfD} \mid \overline{\mathrm{Turnout}}, \mathrm{Age}_{\mathrm{old}}\big)^{(r)} - E\big(\mathrm{AfD} \mid \overline{\mathrm{Turnout}}, \mathrm{Age}_{\mathrm{young}}\big)^{(r)}$$

```
boot_qoi <- boot_betas |>
  mutate(
   ev_young = beta_0 + beta_1 * turnout_mean + beta_2 * age_young + beta_3 * turnout_mean
* age_young,
    ev_old   = beta_0 + beta_1 * turnout_mean + beta_2 * age_old + beta_3 * turnout_mean
* age_old,
    fd       = ev_old - ev_young
  )
boot_qoi
```

OUTPUT

```
# A tibble: 1,000 x 8
   replicate beta_0 beta_1 beta_2  beta_3 ev_young ev_old    fd
       <int>  <dbl>  <dbl>  <dbl>   <dbl>    <dbl>  <dbl> <dbl>
 1         1  -164.  1.22    4.34 -0.0329     6.91   16.3  9.39
 2         2  -136.  0.867   3.76 -0.0254     7.26   16.6  9.31
 3         3  -213.  1.98    5.43 -0.0499     7.06   15.4  8.31
 4         4  -178.  1.34    4.54 -0.0341     6.95   16.9  9.93
 5         5  -211.  1.84    5.37 -0.0465     6.77   16.1  9.34
 6         6  -144.  0.854   3.89 -0.0244     6.71   17.0 10.3
 7         7  -243.  2.21    5.97 -0.0533     6.70   16.4  9.72
 8         8  -256.  2.45    6.37 -0.0600     7.25   16.4  9.19
 9         9  -180.  1.49    4.70 -0.0389     6.76   15.6  8.85
10        10  -376.  4.06    9.17 -0.0976     7.19   16.0  8.84
# i 990 more rows
```

Step 5. The mean and the 2.5% and 97.5% quantiles of each distribution give the estimate and its 95% confidence interval. One long tibble holds all three quantities:

```r
boot_qoi_long <- boot_qoi |>
  pivot_longer(c(ev_young, ev_old, fd), names_to = "quantity", values_to = "value") |>
  mutate(quantity = fct_inorder(quantity))
boot_qoi_long
```

OUTPUT

```
# A tibble: 3,000 x 7
   replicate beta_0 beta_1 beta_2  beta_3 quantity value
       <int>  <dbl>  <dbl>  <dbl>   <dbl> <fct>    <dbl>
 1         1  -164.  1.22    4.34 -0.0329 ev_young  6.91
 2         1  -164.  1.22    4.34 -0.0329 ev_old   16.3
 3         1  -164.  1.22    4.34 -0.0329 fd        9.39
 4         2  -136.  0.867   3.76 -0.0254 ev_young  7.26
 5         2  -136.  0.867   3.76 -0.0254 ev_old   16.6
 6         2  -136.  0.867   3.76 -0.0254 fd        9.31
 7         3  -213.  1.98    5.43 -0.0499 ev_young  7.06
 8         3  -213.  1.98    5.43 -0.0499 ev_old   15.4
 9         3  -213.  1.98    5.43 -0.0499 fd        8.31
10         4  -178.  1.34    4.54 -0.0341 ev_young  6.95
# i 2,990 more rows
```

```r
qoi_summary <- boot_qoi_long |>
  group_by(quantity) |>
  summarize(
    estimate = mean(value),
    lower    = quantile(value, 0.025),
    upper    = quantile(value, 0.975)
  )
qoi_summary
```

OUTPUT

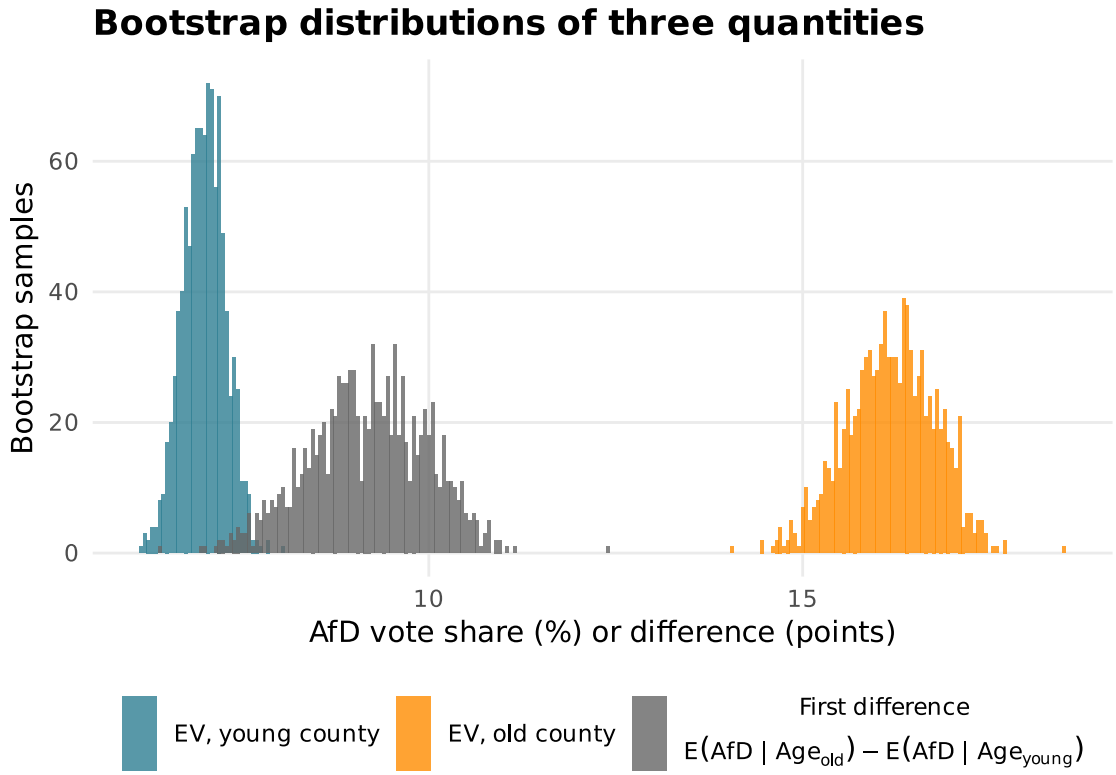
```
# A tibble: 3 x 4
  quantity estimate lower upper
  <fct>       <dbl> <dbl> <dbl>
1 ev_young     7.00  6.46  7.55
2 ev_old      16.2  15.0  17.2
3 fd           9.18  7.58 10.6
```

```r
fd_summary <- qoi_summary |>
  filter(quantity == "fd")
fd_estimate <- fd_summary |>
  pull(estimate)
fd_lower <- fd_summary |>
  pull(lower)
fd_upper <- fd_summary |>
  pull(upper)
```

The three distributions:

```r
qoi_colors <- c(
  ev_young = course_primary,
  ev_old   = "darkorange",
  fd       = "grey40"
)
qoi_labels <- c(
  ev_young = "EV, young county",
  ev_old   = "EV, old county",
  fd       = expression(atop("First difference", E(AfD ~ "|" ~ Age[old]) - E(AfD ~ "|"
~ Age[young])))
)

qoi_plot <- boot_qoi_long |>
  ggplot(aes(x = value, fill = quantity)) +
  geom_histogram(binwidth = 0.05, alpha = 0.8, position = "identity") +
  scale_fill_manual(values = qoi_colors, labels = qoi_labels) +
  labs(
    title = "Bootstrap distributions of three quantities",
    x     = "AfD vote share (%) or difference (points)",
    y     = "Bootstrap samples",
    fill  = NULL
  ) +
  theme_intror()
qoi_plot
```



At average turnout, an old county is expected to vote 9.2 points more AfD than a young one, with a 95% interval from 7.6 to 10.6. This describes counties as they are, not the effect of a county growing older.

Comparison with `marginaleffects`

`predictions()` computes expected values, `comparisons()` first differences. By default they use the delta method, an analytical approximation built on $\hat{\boldsymbol{V}}$. `inferences()` replaces that with the bootstrap (`"boot"`) or with the lecture's draws from the multivariate Normal (`"simulation"`):

```
evs_delta <- predictions(
  age_model,
  newdata = datagrid(turnout = turnout_mean, median_age = c(age_young, age_old)),
  df      = "residual"
)
evs_boot <- evs_delta |>
  inferences(method = "boot", R = n_boot)
evs_simulation <- evs_delta |>
  inferences(method = "simulation", R = n_boot)
evs_boot
```

OUTPUT

```
 turnout median_age Estimate 2.5 % 97.5 %
    76.3       42.4     7.02  6.43   7.52
    76.3       47.6    16.17 14.91  17.36

Type: response
```

```
evs_simulation
```

OUTPUT

```
 turnout median_age Estimate 2.5 % 97.5 %
    76.3       42.4     7.02  6.32   7.71
    76.3       47.6    16.17 15.40  16.93

Type: response
```

```
fds_delta <- comparisons(
  age_model,
  variables = list(median_age = c(age_young, age_old)),
  newdata   = datagrid(turnout = turnout_mean),
  df        = "residual"
)
fds_boot <- fds_delta |>
  inferences(method = "boot", R = n_boot)
fds_simulation <- fds_delta |>
  inferences(method = "simulation", R = n_boot)
fds_boot
```

OUTPUT

```
 turnout Estimate 2.5 % 97.5 %
    76.3     9.15  7.54   10.6

Term: median_age
Type: response
Comparison: 47.562 - 42.448
```

```
fds_simulation
```

OUTPUT

```
 turnout Estimate 2.5 % 97.5 %
    76.3     9.15  8.02   10.4

Term: median_age
Type: response
Comparison: 47.562 - 42.448
```

All in one table, with our own bootstrap first:

OUTPUT

```
# A tibble: 12 x 5
   method                      quantity         estimate lower upper
   <chr>                       <chr>               <dbl> <dbl> <dbl>
 1 Bootstrap by hand           EV, old county      16.2  15.0  17.2
 2 marginaleffects: boot       EV, old county      16.2  14.9  17.4
 3 marginaleffects: simulation EV, old county      16.2  15.4  16.9
 4 marginaleffects: delta      EV, old county      16.2  15.4  16.9
 5 Bootstrap by hand           EV, young county     7.00  6.46  7.55
 6 marginaleffects: boot       EV, young county     7.02  6.43  7.52
 7 marginaleffects: simulation EV, young county     7.02  6.32  7.71
 8 marginaleffects: delta      EV, young county     7.02  6.36  7.68
 9 Bootstrap by hand           First difference     9.18  7.58 10.6
10 marginaleffects: boot       First difference     9.15  7.54 10.6
11 marginaleffects: simulation First difference     9.15  8.02 10.4
12 marginaleffects: delta      First difference     9.15  8.00 10.3
```

> **Note**
>
> All four lie within a few tenths of a point of each other. Bootstrap and simulation don't have to agree: the simulation takes $\hat{\boldsymbol{V}}$ from the OLS formula, which assumes the same error variance for every county. The bootstrap only uses the counties themselves. Where some counties scatter much more than others, its intervals come out wider.

> **A Bayesian parallel**
>
> The five steps resemble how Bayesian analysts work. They draw the coefficients from a *posterior distribution*, which describes how plausible each value of β is given the data. Then they compute every quantity once per draw and summarize the draws, as in Steps 4 and 5. Expected values computed this way are what Bayesian analysts call expected values of the posterior predictive distribution. Adding a draw of the error term, as we do for predicted values below, gives **posterior predictive draws**. Taking the mean of the draws as the point estimate is the posterior mean.
>
> The lecture's draws from $\mathcal{MVN}\left(\hat{\beta}, \hat{\boldsymbol{V}}\right)$ are therefore often called *pseudo-Bayesian*: they are read like a posterior without a Bayesian model behind them. Our bootstrap is the frequentist counterpart. Its draws show how the estimate would vary from sample to sample, not how plausible each β is.

Expected values over a range

Quantity of interest (estimand): the expected AfD share at every turnout value, in the young and in the old county.

As with the lines above, a scenario can run over a whole range: turnout from its minimum to its maximum, in a young and an old county. Step 4 and Step 5 for each of the 50 turnout values:

```
range_scenarios <- expand_grid(
  turnout    = turnout_grid,
  median_age = c(age_young, age_old)
)
```

$$E(\text{AfD} \mid \text{Turnout}, \text{Age})^{(r)} = \tilde{\beta}_0^{(r)} + \tilde{\beta}_1^{(r)}\ \text{Turnout} + \tilde{\beta}_2^{(r)}\ \text{Age} + \tilde{\beta}_3^{(r)}\ \text{Turnout} \cdot \text{Age}$$

```
range_evs <- expand_grid(boot_betas, range_scenarios) |>
  mutate(ev = beta_0 + beta_1 * turnout + beta_2 * median_age + beta_3 * turnout *
median_age) |>
  group_by(turnout, median_age) |>
  summarize(
    estimate = mean(ev),
    lower    = quantile(ev, 0.025),
    upper    = quantile(ev, 0.975),
    .groups  = "drop"
  ) |>
  mutate(scenario = if_else(median_age == age_young, "ev_young", "ev_old"))
range_evs
```

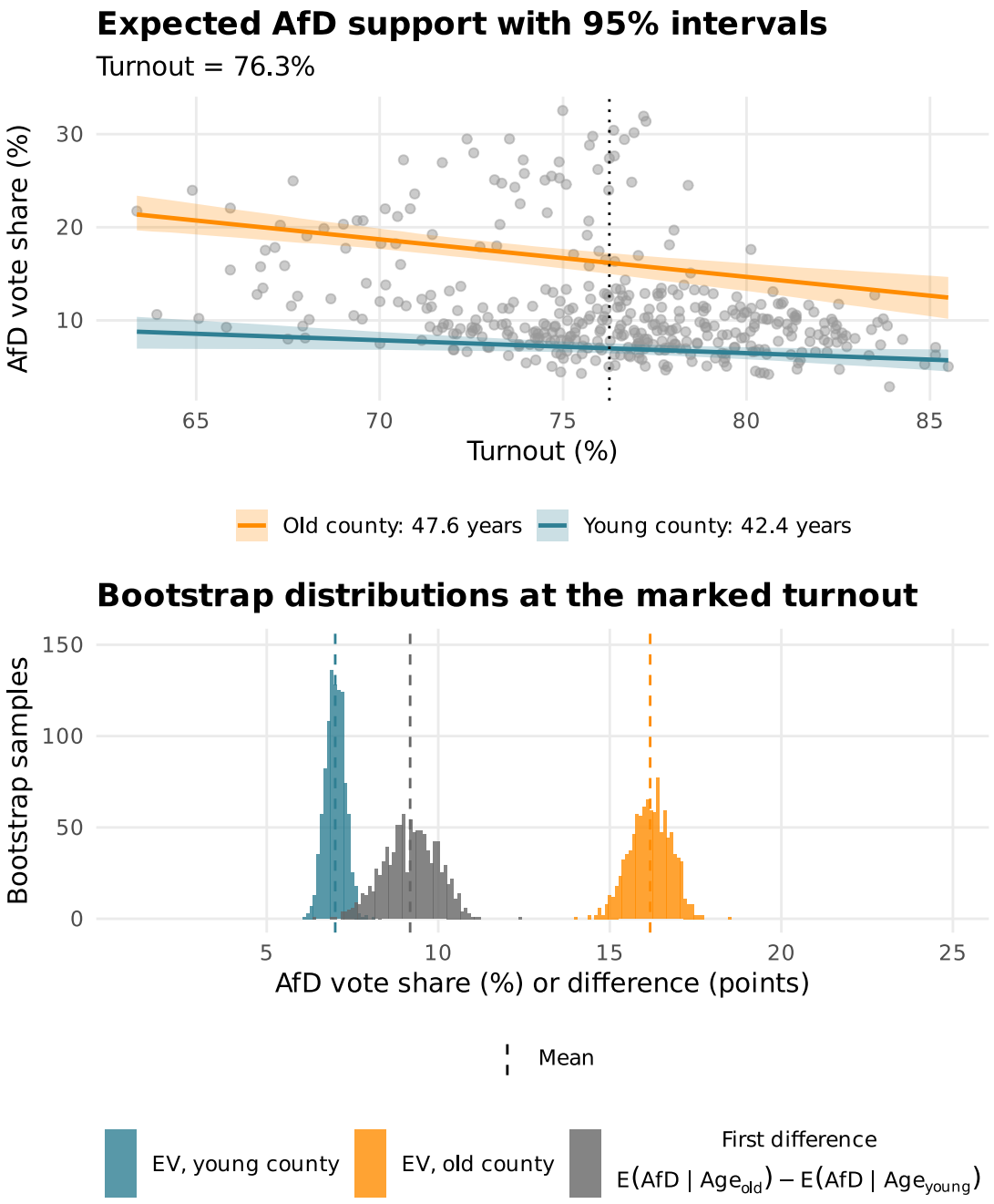
OUTPUT

```
# A tibble: 100 x 6
   turnout median_age estimate lower upper scenario
     <dbl>      <dbl>    <dbl> <dbl> <dbl> <chr>
 1    63.4       42.4     8.78  6.96 10.4  ev_young
 2    63.4       47.6    21.4  19.7  23.4  ev_old
 3    63.8       42.4     8.72  6.96 10.3  ev_young
 4    63.8       47.6    21.2  19.5  23.2  ev_old
 5    64.3       42.4     8.66  6.94 10.2  ev_young
 6    64.3       47.6    21.0  19.4  22.9  ev_old
 7    64.7       42.4     8.59  6.94 10.0  ev_young
 8    64.7       47.6    20.8  19.3  22.7  ev_old
 9    65.2       42.4     8.53  6.93  9.94 ev_young
10    65.2       47.6    20.6  19.2  22.4  ev_old
# i 90 more rows
```

```
range_labels <- c(
  ev_young = str_glue("Young county: {round(age_young, 1)} years"),
  ev_old   = str_glue("Old county: {round(age_old, 1)} years")
)

range_evs_plot <- range_evs |>
  ggplot(aes(x = turnout, color = scenario, fill = scenario)) +
  geom_point(data = counties, aes(y = afd), color = "grey60", fill = NA, alpha = 0.5) +
  geom_ribbon(aes(ymin = lower, ymax = upper), alpha = 0.25, color = NA) +
  geom_line(aes(y = estimate), linewidth = 1) +
 scale_color_manual(values = qoi_colors, labels = range_labels, aesthetics = c("color",
"fill")) +
  labs(
    title = "Expected AfD support with 95% intervals",
    x     = "Turnout (%)",
    y     = "AfD vote share (%)",
    color = NULL,
    fill  = NULL
  ) +
  theme_intror()
```

The bootstrap distributions from above are a cut through this plot at mean turnout, the vertical line. The animation moves the line along the turnout axis, with the distributions at that turnout below:



As the line moves to higher turnout, the old county's expected value falls faster than the young one's, so the first difference shrinks: that is the interaction. The distributions are narrow near average turnout and wide at the edges of the data, where few counties back the estimate.

The same with `marginaleffects`

`predictions()` with the turnout grid and both ages, bootstrapped, gives the bands of `range_evs`. Its estimates come from the original fit, not the mean of the bootstrap samples:

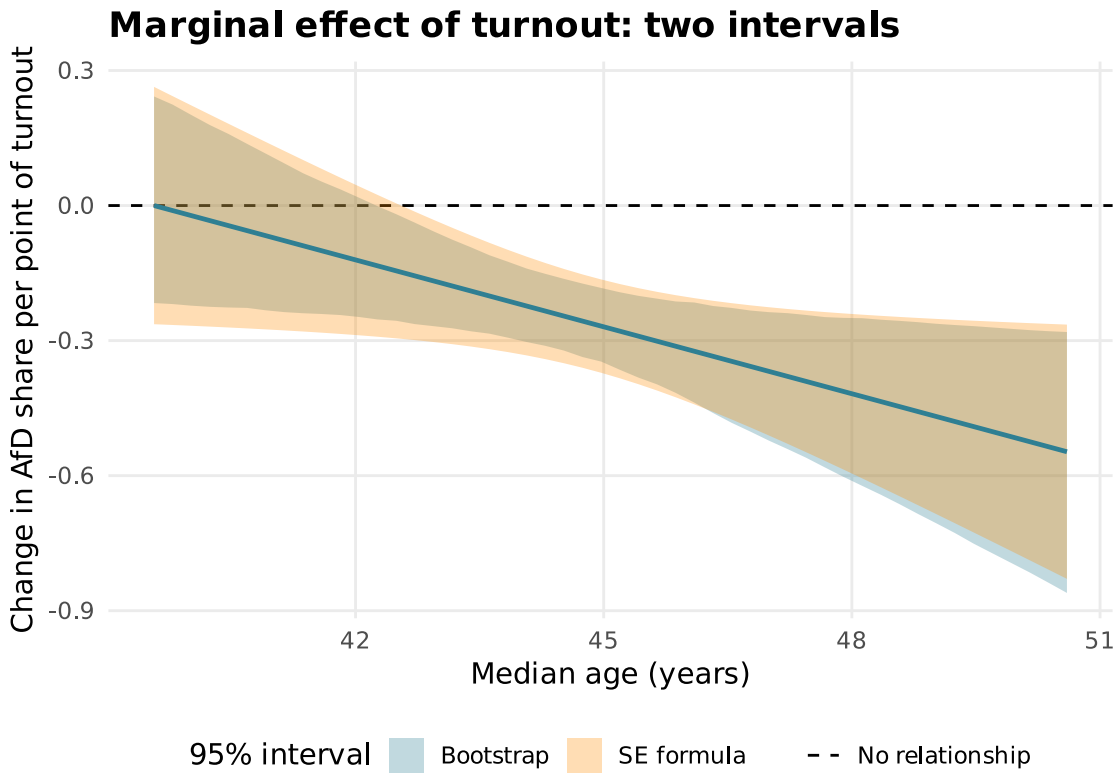
```
predictions(
  age_model,
  newdata = datagrid(turnout = turnout_grid, median_age = c(age_young, age_old))
) |>
  inferences(method = "boot", R = n_boot)
```

The marginal effect of turnout works the same way: compute $\tilde{\beta}_1^{(r)} + \tilde{\beta}_3^{(r)}$ Age for each bootstrap sample and age, then take the quantiles. It reproduces the band from the standard error formula:

```
range_mes <- expand_grid(boot_betas, median_age = age_grid) |>
  mutate(me = beta_1 + beta_3 * median_age) |>
  group_by(median_age) |>
  summarize(
    lower = quantile(me, 0.025),
    upper = quantile(me, 0.975)
  )
```

```r
range_mes_bands <- range_mes |>
  mutate(method = "Bootstrap")
formula_bands <- age_me_curve |>
  select(median_age, lower, upper) |>
  mutate(method = "SE formula")

bind_rows(range_mes_bands, formula_bands) |>
  ggplot(aes(x = median_age)) +
  geom_hline(aes(yintercept = 0, linetype = "No relationship")) +
  geom_ribbon(aes(ymin = lower, ymax = upper, fill = method), alpha = 0.3) +
  geom_line(data = age_me_curve, aes(y = me), color = course_primary, linewidth = 1) +
    scale_fill_manual(values = c("Bootstrap" = course_primary, "SE formula" =
"darkorange")) +
  scale_linetype_manual(values = c("No relationship" = "dashed")) +
  labs(
    title    = "Marginal effect of turnout: two intervals",
    x        = "Median age (years)",
    y        = "Change in AfD share per point of turnout",
    fill     = "95% interval",
    linetype = NULL
  ) +
  theme_intror()
```



The bands differ slightly for two reasons: the bootstrap standard errors are a little larger than the OLS ones (see above), and 1,000 random replicates leave some noise in its quantiles.

The same with `marginaleffects`

`slopes()` over the age grid, bootstrapped, gives the bootstrap band:

```
slopes(
  age_model,
  variables = "turnout",
  newdata   = datagrid(median_age = age_grid)
) |>
  inferences(method = "boot", R = n_boot)
```

Predicted values

Quantity of interest: the AfD share of a single new county with the scenario's values. It is not an estimand in the strict sense: it stays random even with the true β.

A predicted value adds the fundamental uncertainty: each expected value gets a random error from $\mathcal{N}(0, \hat{\sigma}^2)$, with $\hat{\sigma}$ the residual standard deviation, as in the single-predictor example:

```
sigma_hat <- age_model |>
  glance() |>
  pull(sigma)
sigma_hat
```

OUTPUT

```
[1] 4.216819
```

$$\text{PV}^{(r)} = E(\text{AfD} \mid \text{Turnout}, \text{Age})^{(r)} + \tilde{\varepsilon}^{(r)}, \quad \tilde{\varepsilon}^{(r)} \sim \mathcal{N}(0, \hat{\sigma}^2)$$

```
boot_pvs <- boot_qoi |>
  mutate(
    pv_young = ev_young + rnorm(n(), 0, sigma_hat),
    pv_old   = ev_old + rnorm(n(), 0, sigma_hat)
  )
```

```
pv_summary <- boot_pvs |>
  pivot_longer(c(pv_young, pv_old), names_to = "quantity", values_to = "value") |>
  group_by(quantity) |>
  summarize(
    estimate = mean(value),
    lower    = quantile(value, 0.025),
    upper    = quantile(value, 0.975)
  )
pv_summary
```

OUTPUT

```
# A tibble: 2 x 4
  quantity estimate lower upper
  <chr>       <dbl> <dbl> <dbl>
1 pv_old      16.2   8.53  24.4
2 pv_young     7.03 -1.53  15.0
```

`augment()` gives the analytical prediction interval, under the same Normal assumption as our error draws:

```
scenarios <- tibble(
  scenario   = c("young", "old"),
  turnout    = turnout_mean,
  median_age = c(age_young, age_old)
)

age_model |>
  augment(newdata = scenarios, interval = "prediction") |>
  select(scenario, .fitted, .lower, .upper)
```

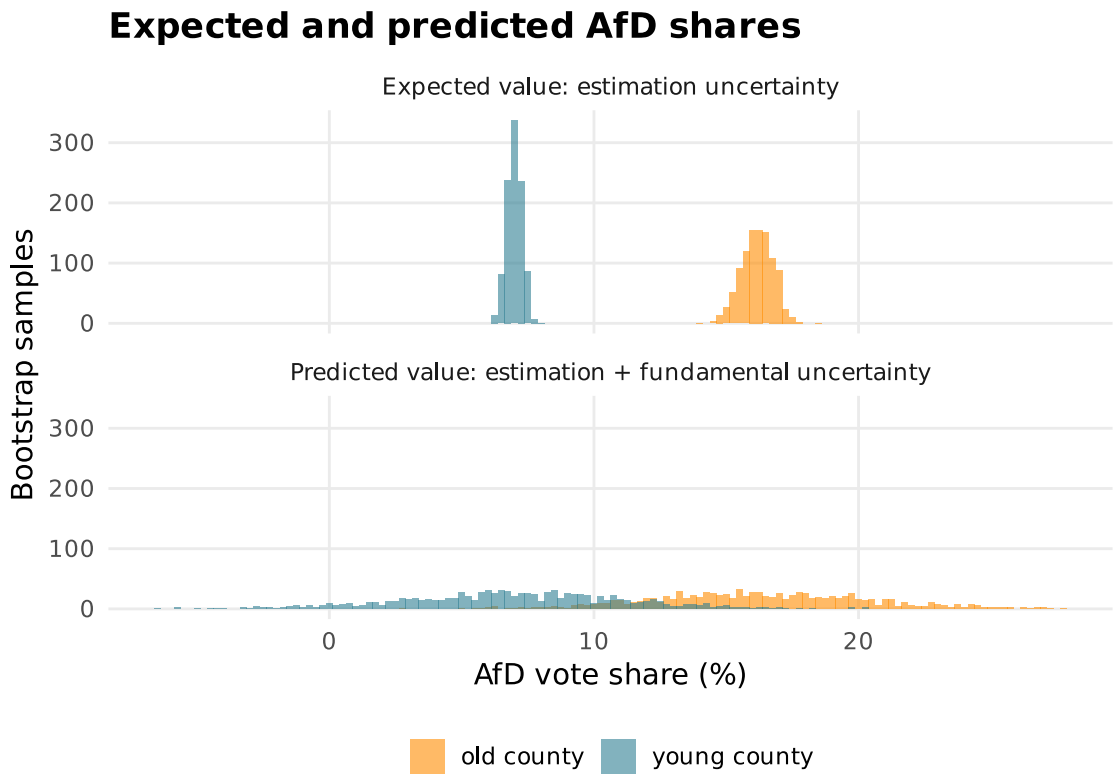
OUTPUT

```
# A tibble: 2 x 4
  scenario .fitted .lower .upper
  <chr>      <dbl>  <dbl>  <dbl>
1 young       7.02  -1.30   15.3
2 old        16.2    7.84   24.5
```

Both give similar intervals. The same scenario, both kinds of values side by side:

```
ev_pv_draws <- boot_pvs |>
  pivot_longer(c(ev_young, ev_old, pv_young, pv_old), names_to = c("type", "scenario"),
names_sep = "_", values_to = "value") |>
  mutate(
    type = if_else(
      type == "ev",
      "Expected value: estimation uncertainty",
      "Predicted value: estimation + fundamental uncertainty"
    ),
    scenario = str_glue("{scenario} county")
  )
```

```r
ev_pv_draws |>
  ggplot(aes(x = value, fill = scenario)) +
  geom_histogram(binwidth = 0.25, alpha = 0.6, position = "identity") +
  facet_wrap(~type, ncol = 1) +
    scale_fill_manual(values = c("young county" = course_primary, "old county" =
"darkorange")) +
  labs(
    title = "Expected and predicted AfD shares",
    x     = "AfD vote share (%)",
    y     = "Bootstrap samples",
    fill  = NULL
  ) +
  theme_intror()
```



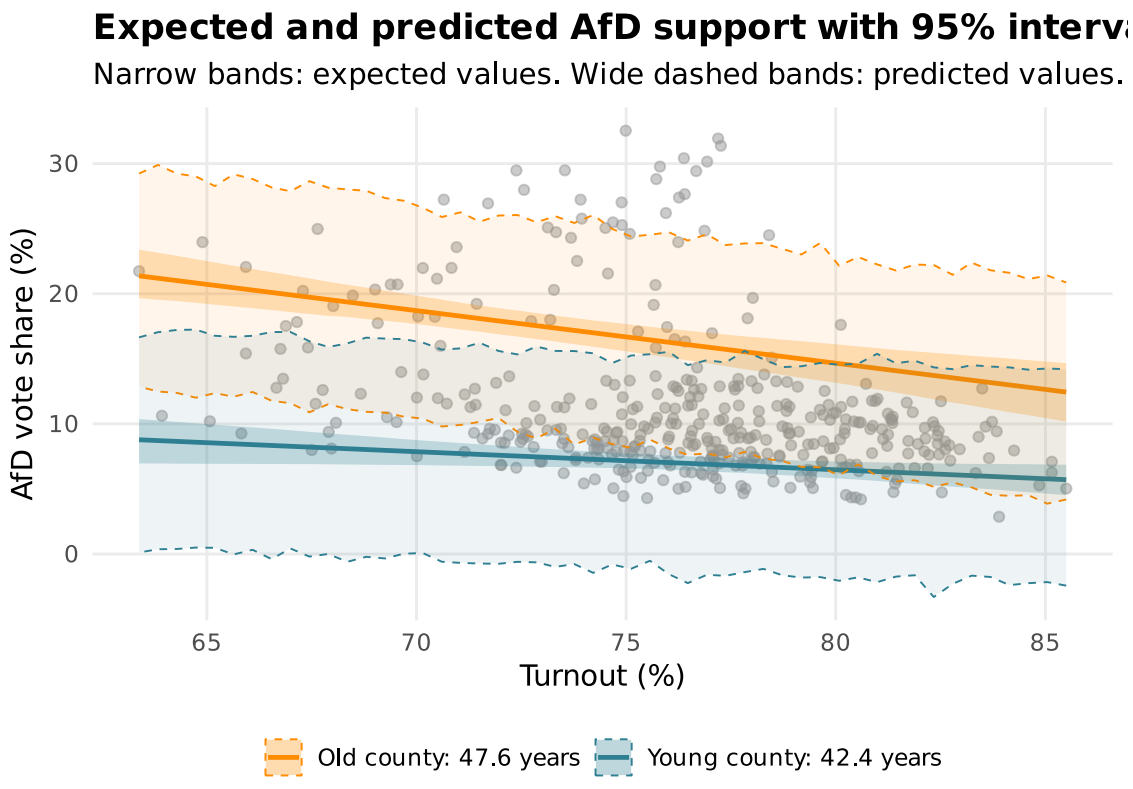
We are fairly sure where the *average* old county lies. A single old county can lie far from it: the predicted values of young and old counties overlap widely.

Only the upper interval is a confidence interval: it is about a fixed quantity, the expected value, and shrinks towards zero width with more data. The lower one is a **prediction interval** for a single new county. With more data it shrinks only to about $\pm 1.96\hat{\sigma}$, the fundamental uncertainty. For the young county it even reaches below zero, as in the single-predictor example.

The same over the whole turnout range: each expected value from `range_evs` gets an error draw, then we take the quantiles.

```r
range_pvs <- expand_grid(boot_betas, range_scenarios) |>
  mutate(
    ev = beta_0 + beta_1 * turnout + beta_2 * median_age + beta_3 * turnout * median_age,
     pv = ev + rnorm(n(), 0, sigma_hat)
  ) |>
  group_by(turnout, median_age) |>
  summarize(
    lower   = quantile(pv, 0.025),
    upper   = quantile(pv, 0.975),
    .groups = "drop"
  ) |>
  mutate(scenario = if_else(median_age == age_young, "ev_young", "ev_old"))
```

```r
range_evs_plot +
  geom_ribbon(data = range_pvs, aes(ymin = lower, ymax = upper), alpha = 0.08, linetype
= "dashed", linewidth = 0.4) +
  labs(
    title    = "Expected and predicted AfD support with 95% intervals",
    subtitle = "Narrow bands: expected values. Wide dashed bands: predicted values."
  )
```



The prediction bands are several times wider than the expected-value bands, and almost as wide in the middle of the turnout range as at its edges: the fundamental uncertainty is the same everywhere, only the estimation uncertainty grows towards the edges.

Concluding remarks

The uncertainty of a marginal effect combines the variances of two coefficients and their covariance; `vcov()` holds all of them. A continuous moderator gives a marginal effect for every value, best shown as a curve with its interval. A DAG has no symbol for moderation: an interaction is part of the model, not of the graph. Expected values, first differences and predicted values turn coefficients into statements in the units of the outcome. The bootstrap gives each its uncertainty with the same five steps, and predicted values add the fundamental uncertainty on top.

Try it: the marginal effect of age

Your task: report the marginal effect of median age on the AfD share in a county with 70% and in one with 80% turnout, each with a 95% bootstrap interval from `boot_betas`. Check both against `slopes()`.

$$\frac{\partial \widehat{\text{AfD}}}{\partial \text{ Age}} = \hat{\beta}_2 + \hat{\beta}_3 \text{ Turnout}$$

```
turnout_low  <- 70
turnout_high <- 80

boot_betas |>
  mutate(
    me_age_low  = beta_2 + beta_3 * turnout_low,
    me_age_high = beta_2 + beta_3 * turnout_high
  ) |>
  pivot_longer(c(me_age_low, me_age_high), names_to = "quantity", values_to = "value")
|>
  group_by(quantity) |>
  summarize(
    estimate = mean(value),
    lower    = quantile(value, 0.025),
    upper    = quantile(value, 0.975)
  )
```

OUTPUT

```
# A tibble: 2 x 4
  quantity    estimate lower upper
  <chr>          <dbl> <dbl> <dbl>
1 me_age_high     1.60  1.18  1.98
2 me_age_low      2.12  1.81  2.46
```

```r
slopes(
  age_model,
  variables = "median_age",
  newdata   = datagrid(turnout = c(turnout_low, turnout_high)),
  df        = "residual"
)
```

OUTPUT

```
 turnout Estimate Std. Error     t Pr(>|t|)     S 2.5 % 97.5 %  Df
      70      2.1      0.155 13.53   <0.001 112.4  1.79   2.40 395
      80      1.6      0.164  9.76   <0.001  65.1  1.28   1.93 395

Term: median_age
Type: response
Comparison: dY/dX
```

Try it: a prediction for Mannheim

Mannheim has the county code `08222`.

Your task: give the AfD share the age model predicts for Mannheim with a 95% interval that includes the fundamental uncertainty, and say whether Mannheim's actual result lies inside it.

```r
mannheim <- counties |>
  filter(county_code == "08222")
mannheim
```

OUTPUT

```
# A tibble: 1 x 7
  county_code state  east region   afd turnout median_age
  <chr>       <chr> <dbl> <chr>  <dbl>   <dbl>      <dbl>
1 08222       08        0 West    9.11    72.2       41.7
```

```r
mannheim_turnout <- mannheim |>
  pull(turnout)
mannheim_age <- mannheim |>
  pull(median_age)

boot_betas |>
  mutate(
      ev = beta_0 + beta_1 * mannheim_turnout + beta_2 * mannheim_age + beta_3 *
mannheim_turnout * mannheim_age,
    pv = ev + rnorm(n(), 0, sigma_hat)
  ) |>
  summarize(
    estimate = mean(pv),
    lower    = quantile(pv, 0.025),
    upper    = quantile(pv, 0.975)
  )
```

OUTPUT

```
# A tibble: 1 x 3
  estimate lower upper
     <dbl> <dbl> <dbl>
1     6.17 -2.10  14.3
```

Try it: first differences in turnout

Your task: in the young and in the old county of our scenario, how much does the expected AfD share change when turnout rises from 72% to 80%? Give both first differences with 95% bootstrap intervals, and the difference between the two first differences with its interval.

```r
turnout_from <- 72
turnout_to   <- 80

boot_betas |>
  mutate(
     ev_young_from = beta_0 + beta_1 * turnout_from + beta_2 * age_young + beta_3 *
turnout_from * age_young,
     ev_young_to    = beta_0 + beta_1 * turnout_to + beta_2 * age_young + beta_3 *
turnout_to * age_young,
     ev_old_from    = beta_0 + beta_1 * turnout_from + beta_2 * age_old + beta_3 *
turnout_from * age_old,
    ev_old_to     = beta_0 + beta_1 * turnout_to + beta_2 * age_old + beta_3 * turnout_to
* age_old,
    fd_young      = ev_young_to - ev_young_from,
    fd_old        = ev_old_to - ev_old_from,
    fd_diff       = fd_old - fd_young
  ) |>
  pivot_longer(c(fd_young, fd_old, fd_diff), names_to = "quantity", values_to = "value")
|>
  group_by(quantity) |>
  summarize(
    estimate = mean(value),
    lower    = quantile(value, 0.025),
    upper    = quantile(value, 0.975)
  )
```

OUTPUT

```
# A tibble: 3 x 4
  quantity estimate lower  upper
  <chr>       <dbl> <dbl>  <dbl>
1 fd_diff     -2.12 -4.07 -0.296
2 fd_old      -3.23 -4.57 -1.96
3 fd_young    -1.11 -2.04 -0.124
```

Check with `marginaleffects`

`comparisons()` gives both first differences. `hypothesis = ~ reference` subtracts the first row (the young county) from the second:

```
comparisons(
  age_model,
  variables = list(turnout = c(turnout_from, turnout_to)),
  newdata   = datagrid(median_age = c(age_young, age_old))
) |>
  inferences(method = "boot", R = n_boot)

comparisons(
  age_model,
  variables  = list(turnout = c(turnout_from, turnout_to)),
  newdata    = datagrid(median_age = c(age_young, age_old)),
  hypothesis = ~ reference
) |>
  inferences(method = "boot", R = n_boot)
```